

GRADO EN INGENIERÍA TELEMÁTICA



VNIVERSITAT
DE VALÈNCIA

TRABAJO FIN DE GRADO

ESTUDIO Y APLICACIÓN DE YOLO, REDES
CONVOLUCIONALES (CNN) Y MODELOS DE
LENGUAJE (LLM) PARA LA DETECCIÓN,
CLASIFICACIÓN Y TRATAMIENTO DE NARANJAS

AUTOR:

JACOBO SAN MARTÍN SÁEZ

TUTOR:

JAUME SEGURA GARCIA



VNIVERSITAT
DE VALÈNCIA



Escola Tècnica Superior
d'Enginyeria **ETSE-UV**

TRABAJO FIN DE GRADO

ESTUDIO Y APLICACIÓN DE YOLO, REDES
CONVOLUCIONALES (CNN) Y MODELOS DE
LENGUAJE (LLM) PARA LA DETECCIÓN,
CLASIFICACIÓN Y TRATAMIENTO DE NARANJAS

AUTOR: JACOBO SAN MARTÍN SÁEZ

TUTOR: JAUME SEGURA GARCIA

Declaración de autoría:

Yo, Jacobo San Martín Sáez, declaro la autoría del Trabajo Fin de Grado titulado “Estudio y aplicación de YOLO, redes convolucionales (CNN) y modelos de lenguaje (LLM) para la detección, clasificación y tratamiento de naranjas” y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores.

Valencia, 25 de junio de 2026

Fdo: Jacobo San Martín Sáez

Resumen:

Las redes neuronales convolucionales han demostrado durante años su eficacia en sectores críticos como la medicina y la ingeniería, a los que se ha sumado recientemente la irrupción de los modelos de lenguaje extensos (LLM), transformando la capacidad de procesamiento de información compleja. Este trabajo tiene como objetivo integrar ambas tecnologías para la detección del estado sanitario de la naranja y la prescripción de tratamientos específicos. Para ello, se van a emplear y comparar las arquitecturas de detección de objetos YOLO, junto con modelos de clasificación implementados en Keras y el modelo de lenguaje Gemini, con el fin de evaluar los resultados y determinar cuál es el más eficiente. En su evaluación para comparar los resultados, se emplearán dos metodologías experimentales: la primera utiliza YOLO como detector inicial de naranjas dañadas para su posterior clasificación mediante Keras (como un detector multiclase de enfermedades que identifica directamente la patología). La otra metodología utiliza solamente YOLO para todo el proceso descrito anteriormente. En ambos casos, el diagnóstico final se vincula con el modelo Gemini para determinar el tratamiento adecuado.

Abstract:

Convolutional neural networks have demonstrated their effectiveness over the years in critical sectors such as medicine and engineering, recently joined by the emergence of large language models (LLMs), which have transformed the capability to process complex information. This study aims to integrate both technologies to detect the health status of oranges and prescribe specific treatments. To achieve this, YOLO object detection architectures will be used and compared alongside classification models implemented in Keras and the Gemini language model, in order to evaluate the results and determine which one is the most efficient. Two experimental methodologies will be employed in the evaluation to compare the results: the first uses YOLO as an initial detector for damaged oranges, followed by classification using Keras (acting as a multiclass disease detector that directly identifies the pathology). The other methodology relies solely on YOLO for the entire process described above. In both cases, the final diagnosis is linked to the Gemini model to determine the appropriate treatment.

Resum:

Les xarxes neuronals convolucionals han demostrat durant anys la seua eficàcia en sectors crítics com la medicina i l'enginyeria, als quals s'ha sumat recentment la irrupció dels models de llenguatge extensos (LLM), transformant la capacitat de processament d'informació complexa. Aquest treball té com a objectiu integrar ambdues tecnologies per a la detecció de l'estat sanitari de la taronja i la prescripció de tractaments específics. Per a fer-ho, s'empraran i es compararan les arquitectures de detecció d'objectes YOLO, juntament com models de classificació implementats en Keras i el model de llenguatge Gemini, amb la finalitat d'avaluar els resultats i determinar quin és el més eficient. En la seua avaluació per a comparar els resultats, s'empraran dues metodologies experimentals: la primera utilitza YOLO com a detector inicial de taronges danyades per a la seua posterior classificació mitjançant Keras (com un detector multiclasse de malalties que identifica directament la patologia). L'altra metodologia utilitza només YOLO per a tot el procés descrit anteriorment. En ambdós casos, el diagnòstic final es vincula amb el model Gemini per a determinar el tractament adequat.

Agradecimientos:

Quiero agradecer a todos aquellos que me han apoyado durante todos estos años: amigos y, sobre todo, mi familia.

Índice general

1. Introducción	17
1.1. Introducción	17
1.2. Motivación	17
1.3. Objetivos	18
1.4. Organización de la memoria	18
1.5. Índice de acrónimos	19
2. Estado del arte	21
2.1. Análisis de aplicaciones similares	22
2.2. Tecnologías	23
3. Requisitos, coste, riesgos, viabilidad y planificación	25
3.1. Requisitos	25
3.1.1. Requisitos funcionales	25
3.1.2. Requisitos no funcionales	26
3.2. Costes	26
3.3. Riesgos	28
3.4. Viabilidad	28
3.5. Planificación	30
4. Análisis	33
4.1. Representación Digital de la Imagen	33
4.1.1. Espacios de Color y Preprocesamiento	34
4.2. Técnicas de Aumento de Datos (Data Augmentation)	35
4.2.1. Transformaciones Espaciales y Geométricas	35
4.2.2. Transformaciones Fotométricas y del Pixel	36
4.3. Dinámica del Aprendizaje y Convergencia Semántica	37
5. Diseño	39
5.1. El paradigma del Aprendizaje Automático	39

5.1.1.	Tipologías de Aprendizaje	39
5.2.	Fundamentos de la Neurona Artificial y el Perceptrón	40
5.2.1.	Estructura computacional de la Neurona	41
5.3.	Evolución y taxonomía de las Redes Neuronales	42
5.4.	Inspiración biológica de las Arquitecturas Convolucionales	42
5.4.1.	Mapeo Funcional del córtex visual en la CNN	43
5.5.	Extracción de características mediante convolución	43
5.5.1.	La operación de convolución	44
5.5.2.	Reducción de dimensionalidad (<i>Pooling</i>)	44
5.6.	El nuevo paradigma: Grandes Modelos de Lenguaje (LLMs)	45
5.6.1.	Operación matemática fundamental: Autoatención Escalada	47
5.6.2.	Más LLM: ChatGPT y Gemini	48
6.	Implementación y pruebas	49
6.1.	Implementación	49
6.1.1.	Análisis de Datos	49
6.1.2.	Configuración y Estructuración de Datasets en Roboflow	49
6.1.3.	Preparación de los Datos antes del entrenamiento	50
6.1.4.	Fase de entrenamiento de los modelos	55
6.1.5.	Tratamiento de enfermedades en las naranjas	59
6.2.	Pruebas	61
6.2.1.	Pruebas Primera Metodología - YOLOv11	61
6.2.2.	Pruebas Primera Metodología - Keras	73
6.2.3.	Pruebas Segunda Metodología - YOLOv11	75
7.	Conclusiones	83
7.1.	Conclusiones	83
7.1.1.	Conclusiones positivas	83
7.1.2.	Conclusiones negativas	87
7.1.3.	Conclusiones finales	87
7.2.	Trabajo futuro	89
	Bibliografía	91

Capítulo 1

Introducción

1.1. Introducción

En 2026, tras la consolidación y evolución exponencial de los modelos de lenguaje de gran escala (LLM), asistimos a un estado de crecimiento e innovación sin precedentes en el campo de la Inteligencia Artificial. Líderes del sector como OpenAI con ChatGPT, Google con Gemini, entre otros desarrolladores, despliegan nuevas versiones y arquitecturas de manera constante, redefiniendo las capacidades de la tecnología.

Sin embargo, este avance de los LLM no ha desplazado en absoluto a las redes neuronales convencionales, las cuales siguen siendo pilares fundamentales en el tejido tecnológico actual debido a su alta funcionalidad y madurez. De hecho, para la ejecución de tareas específicas como la visión artificial, la clasificación de patrones biomédicos o el procesamiento de señales en tiempo real, suelen superar a los propios modelos de lenguaje generales.

Mientras que los LLM destacan por su versatilidad, capacidad de razonamiento abstracto y comprensión del contexto global, a menudo adolecen de un coste computacional elevado y tiempos de latencia críticos. Por el contrario, las redes neuronales especializadas ofrecen una precisión milimétrica, una eficiencia de recursos óptima y una fiabilidad determinista en entornos acotados. Por ello, en este paradigma tecnológico actual no busca la elección de una tecnología u otra, sino la convergencia: aprovechar la robustez de las redes tradicionales en la fase de análisis específico y la flexibilidad de los LLMs para la interpretación y generación de soluciones complejas.

Por ello, en este estudio se plantea la aplicación y combinación de estas tecnologías para el desarrollo de un sistema capaz de detectar y tratar la enfermedad de la naranja.

1.2. Motivación

La IA ha dejado de ser una tecnología emergente para convertirse en un motor de disrupción profunda con un impacto medible en industrias y corporaciones de escala global. En los últimos años, el despliegue de arquitecturas basadas en agentes de IA y la democratización de modelos avanzados han transformado radicalmente los modelos de negocio y la automatización de procesos.

Este suceso despertó mi interés académico y profesional sobre cómo articular, de ma-

nera estratégica e inteligente dichas tecnologías para resolver problemas del mundo real. Más allá de la adopción genérica de la IA, este trabajo nace de la inquietud por explorar, orquestar y combinar estos modelos especializados para ofrecer soluciones precisas y eficientes.

Además, desde siempre he tenido una gran curiosidad en como clasificar distintos datos para resolver un determinado problema de la manera más óptima posible.

1.3. Objetivos

La finalidad principal de este estudio consiste en diseñar un sistema que ejecute un proceso de análisis y diagnóstico para identificar posibles patologías y, en caso afirmativo, dar un tratamiento adecuado. Todo esto aplicando dos metodologías distintas. Para ello se llevarán una serie de pasos clave.

- Diseñar y entrenar dos modelos, un YOLO (CNN) para clasificar en buen estado o mal estado la naranja para usarlo en nuestro modelo de Keras (CNN) donde a partir de nuestras naranjas en mal estado determinar que tipo de enfermedad presenta para usarlo en nuestra primera metodología.
- Diseñar y entrenar un modelo de YOLO (CNN) capaz de detectar la enfermedad de la naranja para usarlo en nuestra segunda metodología.
- Evaluar y optimizar el rendimiento de los modelos utilizando los parámetros más adecuados para asegurar su robustez y eficiencia en diversas condiciones.
- Emplear el modelo Gemini a través de una clave de API para determinar y prescribir el tratamiento correspondiente para usarlo en las dos metodologías.

1.4. Organización de la memoria

La organización de la memoria se puede estructurar en siete bloques principales incluyendo este capítulo de introducción.

1. En el capítulo primero, *Introducción*, En esta sección se introduce el proyecto y se explica por qué ha sido realizado, así como sus objetivos.
2. En el capítulo segundo, *Estado del arte*, se comparan y explican el momento en el que se encuentran este tipo de proyectos y las tecnologías que emplean. También se contextualizan y relacionan con un poco de historia.
3. En el tercer capítulo se detallan los costes previstos y se abordan aspectos como la viabilidad y los riesgos del proyecto. Se ofrece una revisión de la proyección y las estimaciones iniciales sobre el desarrollo del proyecto.
4. En el capítulo cuarto, *Análisis*, se profundiza en los fundamentos teóricos y matemáticos de la representación digital de imágenes. Asimismo, se detallan diferentes técnicas de optimización de datos y entrenamiento.

5. En el quinto capítulo, sección *Diseño*, se explica la historia sobre el aprendizaje automático, sus tipologías, fundamentos, evolución y operación matemática de convolución.
6. En el sexto capítulo, *Implementación*, se explica su implementación y el proceso de pruebas que se han hecho durante este estudio.
7. Por último, en el capítulo *Conclusiones*, se presenta una reevaluación del proyecto una vez desarrollado, así como las líneas de trabajo futuro para el mismo.

1.5. Índice de acrónimos

- **API**: Interfaz de Programación de Aplicaciones
- **COCO**: Objetos Comunes en Contexto
- **CNN**: Red Neuronal Convolutiva
- **IA**: Inteligencia Artificial
- **LLM**: Modelo de Lenguaje de gran escala
- **RGB**: Rojo, verde, azul
- **RT-DETR**: Real-Time DEtection TRansformer
- **RNN**: Red Neuronal Recurrente
- **YOLO**: You Only Look Once (un tipo de CNN)

Capítulo 2

Estado del arte

En el campo agrícola, se ha apostado por tecnologías de Machine Learning durante bastantes años. La monitorización y gestión de los cultivos ha experimentado una transición bastante llamativa, pasando de los sistemas informáticos basados en la Visión por Computadora Clásica a principios de los años 2000, los cuales dependían de la extracción manual de características geométricas y de color, a las ya familiares arquitecturas nombradas anteriormente que dominan la actualidad.

La IA es ahora la encargada de la detección temprana de plagas y la gestión del rendimiento de las cosechas, lo que ha supuesto una democratización tecnológica en un mercado donde antes era muy complejo implementar automatizaciones sin disponer de grandes infraestructuras e inversiones. Hoy en día, la agricultura de precisión es cada vez más accesible tanto para pequeños productores como para la gran industria agroalimentaria.

Pero no es solo el análisis estático de los campos el que ha dado un cambio radical, la toma de decisiones agronómicas y la prescripción de tratamientos fitosanitarios ha comenzado a ser también responsabilidad compartida con las máquinas. En el artículo [1], se analiza cómo las técnicas de aprendizaje profundo, lideradas por las redes neuronales convolucionales (CNN) como estándar industrial, revolucionaron el sector; las CNN destacaban ya en la extracción de características morfológicas y cromáticas de los frutos, mientras que se vislumbraba el potencial de las redes neuronales recurrentes (RNN) en el análisis de series temporales para la predicción de cosechas.

Hoy en día, partiendo de esa base consolidada, el estado de la técnica ha evolucionado hacia arquitecturas de detección en tiempo real como YOLO y modelos basados en Transformers (como RT-DETR), optimizando drásticamente la velocidad y la precisión en entornos de campo dinámicos. Como se analiza en la revisión de [2], la evolución de las técnicas de visión artificial en la agricultura ha permitido transicionar desde las primeras CNN de clasificación básica hacia modelos CNN más actuales como YOLO de detección multiclase en tiempo real, optimizando la precisión en la identificación de patologías en entornos de campo abiertos. Asimismo, el estudio de [3] destaca el cambio de paradigma actual hacia los Vision Transformers en el sector agrícola, demostrando que arquitecturas basadas en mecanismos de atención ofrecen un rendimiento superior a las CNN clásicas y en algunos casos compitiendo con YOLO al capturar variaciones sutiles de color y textura provocadas por plagas y enfermedades.

2.1. Análisis de aplicaciones similares

En este proyecto, para la fase de etiquetado y preparación del conjunto de datos, se ha seleccionado la plataforma Roboflow [4], una herramienta líder en el flujo de trabajo de visión por computadora. Si bien existen alternativas de código abierto consolidadas como Label Studio [5], que ofrecen una gran flexibilidad para el etiquetado multimapa, Roboflow presenta ventajas competitivas críticas. La principal de ellas es su amplia variedad de datasets públicos. Esta característica no solo optimiza el tiempo de desarrollo mediante el aprovechamiento de imágenes ya anotadas por la comunidad, sino que también facilita la transferencia de aprendizaje y el aumento de datos unificado dentro de la misma plataforma, superando las limitaciones operativas de herramientas de etiquetado local tradicionales. También ofrece herramientas de asistencia de IA integradas (como SAM-2 para segmentación), análisis de conjuntos de datos, búsqueda semántica, aumento de datos y capacidad de entrenamiento y despliegue de modelos directamente en la plataforma, lo que reduce significativamente la fricción para equipos no técnicos o aquellos que priorizan la velocidad de implementación.

En este trabajo, en la fase de entrenamiento de redes neuronales se han implementado los modelos YOLO [6] y el Keras [7]. El propósito de integrar ambas tecnologías no es utilizarlas de forma aislada, sino evaluar y contrastar dos metodologías de diseño distintas para la resolución del problema. A pesar del auge reciente de los modelos basados en Vision Transformers (ViTs) [8], como el Detection Transformer (DETR), en este proyecto se ha descartado explícitamente su implementación debido a su elevado coste computacional y de infraestructura. Los Transformers sustituyen las operaciones de convolución locales por mecanismos de autoatención global (self-attention), lo que les permite capturar dependencias de largo alcance en la imagen con una precisión sobresaliente. Sin embargo, esta capacidad introduce una complejidad computacional cuadrática respecto al tamaño de la entrada ($O(N^2)$, donde N es el número de píxeles de la imagen).

En otros muchos proyectos Roboflow ha sido una plataforma eficiente a la hora de realizar datasets, en el contexto de este artículo [9], el uso de Roboflow Universe no es un mero detalle en la metodología, sino el verdadero motor que hace viable un proyecto de esta envergadura.

En la inteligencia artificial, existe un dicho: *“Un modelo es tan bueno como los datos con los que se le entrena”*. Para los autores de YOLO-Pest, enfrentarse al reto de detectar insectos diminutos como las moscas de la fruta significaba que el software tradicional de etiquetado se quedaría corto. Aquí es donde Roboflow se convierte en el protagonista silencioso de la investigación. Etiquetar miles de imágenes de insectos milimétricos en entornos agrícolas (donde se confunden con hojas, tierra o ramas) es una pesadilla logística. Esta plataforma actúa como el centro neurálgico del proyecto, permitiendo a los investigadores organizar, filtrar y auditar la calidad de las imágenes para asegurar que no haya errores de asignación antes de que el modelo vea una sola foto.

Como continuación directa de esta tendencia metodológica, el trabajo de Hemalatha et al. [10] consolida la relevancia de este enfoque al proponer una estrategia innovadora para la detección de plagas en cultivos de pepino. En dicha investigación, los autores validan la efectividad de Roboflow como la herramienta fundamental para el proceso de anotación y estructuración de imágenes, explotando su versatilidad para exportar los datos al formato estándar COCO y alimentar algoritmos de la familia YOLO. Este flujo de trabajo demuestra que la centralización del pipeline de datos en plataformas en la nube no es un caso aislado, sino un estándar emergente para optimizar la detección en

escenarios de alta complejidad biológica.

Evidentemente, no se debe despreciar el trabajo que realizan las redes neuronales en este escenario, ya que sin un predictor robusto y optimizado capaz de generalizar sobre dichos datos, el esfuerzo depositado en la preparación del dataset carecería de utilidad práctica. Mientras que plataformas como Roboflow garantizan la integridad y la calidad de la información de entrada, son arquitecturas como YOLO y los clasificadores basados en Keras las encargadas de asimilar esos patrones visuales complejos y precisos sobre el fruto, terreno o lo que queramos clasificar. Al fin y al cabo, la ingeniería de datos y la optimización de modelos no son etapas independientes, sino el éxito del sistema en entornos agrícolas reales radica en el equilibrio perfecto entre una segmentación de datos impecable y una capacidad de cómputo eficiente.

En conclusión, estos artículos son un claro ejemplo de cómo una gestión de datos impecable y accesible es el factor determinante para llevar la inteligencia artificial del laboratorio al campo agrícola. Mientras que las redes neuronales proporcionan la capacidad computacional para el procesamiento y aprendizaje, Roboflow actúa como la plataforma de orquestación y gestión del ciclo de vida del dataset.

Por último, para la fase de diagnóstico y propuesta de tratamientos específicos contra las enfermedades de los cítricos, se ha integrado el modelo de Gemini [11] a través de su interfaz de programación de aplicaciones (API). En el panorama actual de la inteligencia artificial generativa existían diversas alternativas competitivas, tales como DeepSeek o ChatGPT (OpenAI); sin embargo, la selección de Gemini responde a criterios estrictos de rendimiento técnico y viabilidad económica. Respecto a DeepSeek, al contrastar las capacidades de procesamiento, Gemini tiene un rendimiento superior. Y respecto a ChatGPT, los modelos comerciales de OpenAI ofrecen capacidades ligeramente mejor que Gemini, pero su implementación requería un modelo de suscripción mensual de pago para uso de tokens. En este artículo se reflejan las comparativas escritas: [12] .

Aunque no hayan muchos artículos o estudios que concretamente tengamos un claro ejemplo parecido a este trabajo, abajo en la tabla de tecnologías 2.1 si hay un estudio que presenta la utilización de Gemini.

2.2. Tecnologías

Para el código de este proyecto, se ha seleccionado Python 3.11 [13] como lenguaje de programación. Esta elección es la más común para entrenamiento de inteligencia artificial, además, responde a criterios críticos de rendimiento y compatibilidad tecnológica dentro del ecosistema del aprendizaje profundo. Por otro lado, ofrece un soporte maduro y nativo para las versiones estables de las librerías de este proyecto.

La implementación y el proceso de optimización de los modelos se han llevado a cabo mediante TensorFlow [14], un framework de código abierto desarrollado por Google que se consolida como uno de los estándares más robustos en el ecosistema del aprendizaje automático. Esta plataforma destaca por su alta eficiencia en el diseño y despliegue de soluciones de inteligencia artificial, ofreciendo una escalabilidad excepcional que permite trasladar modelos de manera fluida desde dispositivos móviles hasta infraestructuras distribuidas de producción masiva.

Dentro de TensorFlow, el desarrollo de una de las redes neuronales se ha articulado a través de Keras [7]. Keras actúa como una capa de abstracción que simplifica significativa-

mente la construcción de arquitecturas complejas gracias a una sintaxis limpia, intuitiva y modular. Presenta una sencilla estructura de división de clases dividiéndose en carpetas, a diferencia de otros flujos de trabajo que dependen de plataformas externas de anotación como Roboflow o Label Studio, la metodología implementada en Keras aprovecha el propio sistema de archivos local para la estructuración y etiquetado automatizado del conjunto de datos.

Por otra parte, se ha usado la librería Ultralytics [15], concretamente YOLO. Se apoya en la suite de código abierto, empleando específicamente la arquitectura YOLO (You Only Look Once). Ultralytics se ha consolidado como el entorno de referencia para la ejecución de modelos de la familia YOLO, abstrayendo la complejidad operativa asociada al entrenamiento y despliegue de detectores de objetos en tiempo real.

A continuación, se ha usado la biblioteca dotenv [16]: empleada para cargar de forma segura la clave de interfaz de programación de aplicaciones (API key) de Google almacenada en las variables de entorno locales. Esta práctica de ingeniería de software garantiza la protección de los tokens de acceso y la confidencialidad de las credenciales de desarrollo. Y también, la librería google.generativeai [11]: utilizada como la interfaz directa para interactuar con el modelo fundacional Gemini. A través de este paquete, se gestionan las peticiones, la configuración de los parámetros de generación y el consumo de tokens asignados al modelo de lenguaje multimodal de Google para la ejecución de las tareas correspondientes del proyecto.

Por otro lado, para mitigar el riesgo de sobreentrenamiento (overfitting) y mejorar la capacidad de generalización de los modelos, se ha integrado la librería Albumentations [17]. Esta herramienta es prácticamente un estándar en el campo de detección de objetos debido a su alta eficiencia y a su extensa variedad de técnicas de transformación de imágenes, que incluyen desenfoque gaussiano, modificaciones de brillo y contraste, recortes aleatorios (cropping), rotaciones e inversiones espaciales (flips).

Y por último, para la representación gráfica de las métricas, matrices de confusión y la inspección visual de los resultados, se ha empleado la biblioteca Matplotlib [18] y Sklearn [19]. Matplotlib constituye uno de los estándares de facto en el ecosistema científico de Python gracias a su versatilidad y solidez, luego Sklearn ha servido para sacar resultados sobre matrices de confusión y reportes de clasificación, muy importantes en este proyecto para sacar constantemente conclusiones sobre los diferentes resultados.

Aquí se ha elaborado una tabla, 2.1 , que muestra la presencia de estas tecnologías en otros trabajos similares.

Referencia	YOLO	Keras	TensorFlow	Albumentations	Gemini API
[20]	Sí	No	No	Sí	No
[21]	Sí	No	No	No	No
[22]	No	No	No	No	Sí
[23]	No	Sí	Sí	No	No
[24]	Sí	No	No	No	No

Cuadro 2.1: PRESENCIA DE TECNOLOGÍAS EN OTROS TRABAJOS SIMILARES

Capítulo 3

Requisitos, coste, riesgos, viabilidad y planificación

3.1. Requisitos

Los requisitos funcionales especifican el comportamiento de un sistema, las tareas concretas y el comportamiento que el sistema debe ejecutar para satisfacer los objetivos del proyecto. La correcta determinación de estos requisitos exige una fase previa, la cual contempla tanto un análisis exhaustivo de los flujos de desarrollo como el levantamiento detallado de las necesidades, restricciones y expectativas del cliente.

3.1.1. Requisitos funcionales

- El sistema debe permitir al usuario especificar la ruta de un directorio local con imágenes de cultivos de cítricos para su procesamiento.
- El sistema debe ser capaz de procesar las imágenes mediante modelos de la versión YOLOv11 para localizar y delimitar mediante el etiquetado correspondiente los ejemplares afectados por patologías.
- El sistema para su primera metodología debe permitir un flujo secuencial donde YOLOv11 realiza una detección binaria (naranja buena/mala), extrae las regiones de interés (*crops*) de las detecciones anómalas, y las deriva al Keras para su clasificación.
- El sistema para su primera metodología debe estructurar las diferentes carpetas de forma automática, moviendo cada imagen o recorte a la subcarpeta correspondiente según la patología diagnosticada (en el CNN de Keras).
- El sistema para su segunda metodología debe permitir un flujo directo donde un modelo YOLOv11 multiclase identifica y clasifica la enfermedad específica de la naranja directamente sobre la imagen original en un solo paso.
- El sistema para sus dos metodologías debe conectarse con la API de Google Gemini para pasarle la ruta de una imagen de los defectos junto con un prompt especializado, devolviendo un diagnóstico y una recomendación de tratamiento.

- El sistema debe ser capaz de evaluar el rendimiento de ambos enfoques generando de forma automatizada reportes de clasificación detallados (*precisión*, *recall*, F1-score) y matrices de confusión visuales mediante Matplotlib y Scikit-Learn.

3.1.2. Requisitos no funcionales

- El sistema debe configurarse para ejecutar el código aprovechando la aceleración por hardware local (GPU).
- El correcto funcionamiento de la API de Gemini queda condicionado a la disponibilidad de conexión a internet y a la estabilidad del servicio en la nube.
- El recall de los 3 modelos en la clasificación debe ser de al menos un 70 % de media.
- El código debe estar debidamente documentado, y con manejo de excepciones, para facilitar la legibilidad del código, el mantenimiento y la actualización de los modelos de IA.

3.2. Costes

El cronograma del proyecto en un principio contempló un periodo comprendido entre principios de agosto de 2025 y diciembre de ese mismo año, sin embargo, en medio del desarrollo del proyecto se tuvo que cambiar de planificación y rehacer el diagrama de Gantt como se puede ver en 3.2 en comparación a 3.3. Una vez quedando reflejado esto, se da paso al cronograma del proyecto final donde contempla un periodo comprendido entre principios de agosto y finales de junio de 2026, acumulando un total estimado de 350 horas de dedicación. Con el objetivo de cumplir con los plazos establecidos y optimizar el desarrollo de las tareas, la distribución temporal se ha estructurado bajo un régimen de carga diaria de una hora y media, contados exclusivamente de lunes a viernes y excluyendo los fines de semana.

Costos Temporales

- **Duración del proyecto:** Desde inicios de agosto hasta finales de junio del siguiente año (aproximadamente 10 meses).
- **Total de horas planificadas:** 350 horas.
- **Horas de trabajo diarias:** 1 hora y 30 minutos por día hábil.

Costos Económicos

- **Coste por hora de desarrollo:** Suponiendo un costo de 20 € por hora de desarrollo, el coste total sería:

$$350 \text{ horas} \times 20\text{€/hora} = 7,000\text{€}$$

Infraestructura y Herramientas

La infraestructura tecnológica empleada durante las fases de entrenamiento y validación constituye un factor determinante en el éxito de los proyectos de visión por computadora y aprendizaje profundo. Disponer de una alta capacidad de cómputo agiliza los ciclos de prueba y testeo, lo que incrementa significativamente las posibilidades de converger hacia una solución con el máximo rendimiento posible.

En este trabajo, el desarrollo se ha llevado a cabo utilizando un equipo informático local de gama de entrada con un valor de mercado aproximado de 500 €. A pesar de tratarse de un hardware limitado frente a las estaciones de trabajo industriales, su elección responde a las restricciones de recursos disponibles, convirtiéndose en un escenario idóneo para evaluar la viabilidad de los modelos.

A nivel de componentes esenciales, se aprovecha una tarjeta gráfica dedicada NVIDIA, donde aunque sea una tarjeta gráfica descatalogada y de hace años sigue siendo más eficiente que AMD cuya arquitectura permite explotar la tecnología de aceleración por hardware CUDA y AMD no (si es cierto que en algunas tarjetas gráficas de AMD pueden soportar CUDA, pero no de forma nativa, por eso en este proyecto era mejor no arriesgarse a pesar de tener una gráfica de AMD más potente).

Componente	Especificación
Procesador	Intel i7 8700k
Placa Base	TUF Z370-PRO GAMING
Memoria RAM	Corsair Vengeance 32 GB (2 x 16 GB) DDR4-3200 MHz
Tarjeta Gráfica	NVIDIA Geforce GTX 1070 Ti 8GB VRAM
Almacenamiento SSD	KINGSTON SNV2S500G
Disco Duro Adicional 1	Samsung SSD 960 EVO 500GB
Disco Duro Adicional 2	WDC WD60EFRX-68L0BN1
Sistema Operativo	Windows 10 Pro 64-bit

Costos Detallados

Concepto	Costo (€)
Desarrollo (350 horas)	7,000
Infraestructura (PC)	500
Documentación y Mantenimiento	500
Total Estimado	8,000

Cuadro 3.1: COSTOS DEL PROYECTO EN EUROS

Concepto	Costo Mensual
Vivienda	400 €
Luz	65 €
Agua	40 €
Internet	25 €

Cuadro 3.2: COSTOS MENSUALES DE LA VIVIENDA

3.3. Riesgos

En el contexto del análisis y clasificación de enfermedades en los cítricos, debemos considerar diferentes problemas que podrían influir negativamente en el rendimiento y entrenamiento de los modelos.

Los riesgos son los siguientes:

- Variabilidad en la interpretación de características de las etiquetas.
- Complejidad y diversidad de enfermedades presentadas en las naranjas.
- Insuficiencia en la cantidad de imágenes de entrenamiento.
- Sensibilidad de calidad de la imagen de entrada.

3.4. Viabilidad

La razón principal por la que consideramos el proyecto como viable es que el uso de YOLO para la detección y clasificación de cítricos es una técnica que ya ha demostrado tener un buen uso en otros trabajos. Además, Roboflow ha sido el conjunto escogido en innumerables estudios en tareas de visión por computadora de aprendizaje automático.

Por otro lado, los factores clave para confiar en la viabilidad del sistema aquí presentado son su aplicación práctica y su alta adaptabilidad. Una vez completado el diseño y entrenamiento de las CNNs, existe la posibilidad de exportar el modelo resultante e implementarlo en diferentes aplicaciones de inspección o sistemas automatizados de forma sencilla. Por otra parte, en caso de recibir una petición personalizada para detectar patologías o anomalías específicas en otros cultivos, una opción directa a considerar sería el reentrenamiento de la redes neuronales utilizando un nuevo conjunto de datos etiquetado.

DAFO

En esta figura 3.1 se muestra el análisis DAFO del proyecto. DAFO (acrónimo de Debilidades, Amenazas, Fortalezas y Oportunidades) constituye una herramienta analítica fundamental para la evaluación de un proyecto o idea, cuyo propósito principal es la identificación sistemática de sus riesgos latentes y sus cualidades intrínsecas. La implementación de este análisis resulta determinante en el diseño de la planificación metodológica, ya que permite maximizar el impacto de las fortalezas y el aprovechamiento de las oportunidades, al tiempo que se mitigan los efectos de las amenazas externas y las debilidades operacionales.

Dada su versatilidad, este marco de evaluación no debe concebirse como un diagnóstico estático, sino como un proceso dinámico que requiere revisiones periódicas en hitos clave del ciclo de vida del software. Llevar a cabo un seguimiento continuo a lo largo del periodo de desarrollo proporciona una base empírica de gran utilidad para la adaptación, el control de cambios y el ajuste preciso tanto de los requisitos funcionales como de las especificaciones técnicas del sistema.

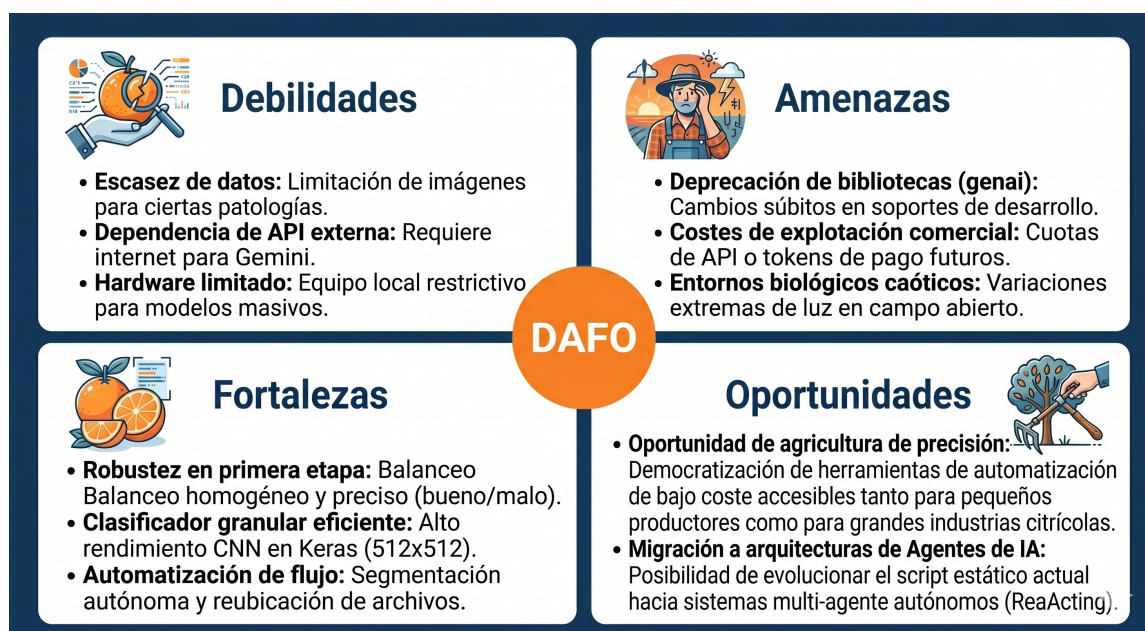


Figura 3.1: DAFO

3.5. Planificación

La planificación y ejecución de este proyecto se ha articulado bajo el marco de trabajo ágil Scrum, una metodología idónea para el desarrollo de sistemas de software complejos gracias a su naturaleza iterativa e incremental basada en fases de trabajo denominadas sprints. Al inicio y cierre de cada ciclo se llevan a cabo sesiones de revisión y retrospectiva, orientadas a evaluar los avances logrados, mitigar posibles desviaciones y planificar de forma sistemática el siguiente bloque de objetivos, garantizando de este modo un proceso de mejora continua.

Al comienzo del desarrollo, se diseñó un cronograma inicial detallado mediante un diagrama de Gantt (representado en la figura 3.2), el cual definía de forma estricta los plazos estimados para cada una de las tareas del proyecto. No obstante, a lo largo del ciclo de vida del desarrollo surgieron desviaciones en la ruta crítica debido a la complejidad técnica de ciertas fases previas, lo que provocó retrasos y obligó a replantear los marcos temporales.

A consecuencia de ello, se elaboró una planificación actualizada (detallada en la figura 3.3), donde se extendieron los plazos de entrega y se reajustaron las fechas definitivas de presentación debido a desfases temporales acumulados.

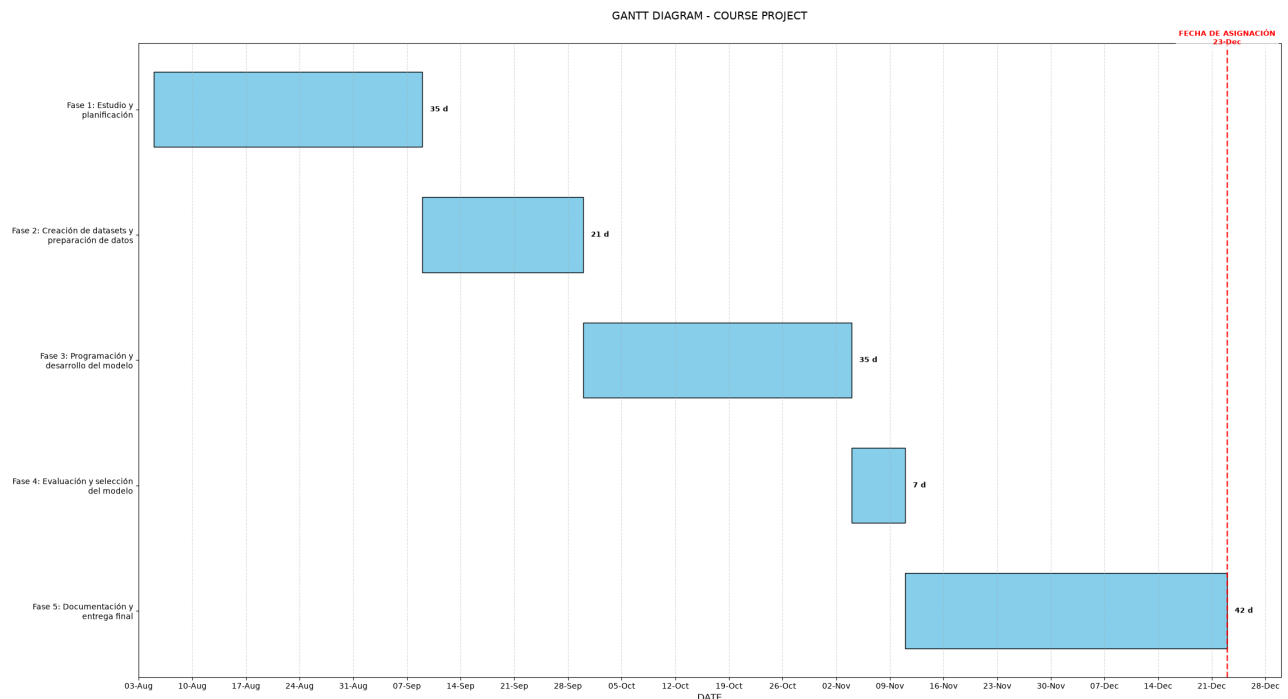


Figura 3.2: DIAGRAMA DE GANTT INICIO DEL PROYECTO

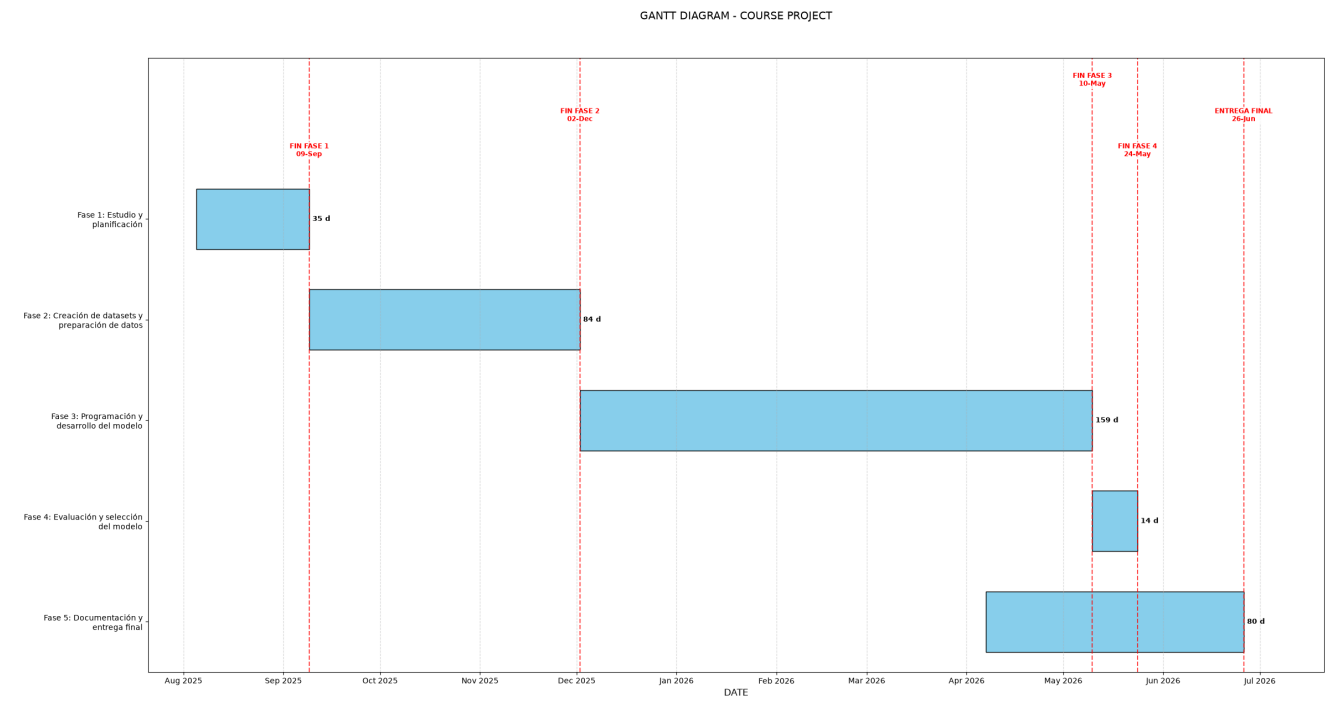


Figura 3.3: DIAGRAMA DE GANTT DESPUÉS DE REALIZAR EL PROYECTO

Fases del proyecto

En los diagramas de Gantt se ve reflejado el proyecto dividido en 5 fases. A continuación se explicarán en que consiste cada fase del proyecto.

Fase 1: Estudio y planificación

Esta etapa inicial constituye la base teórica y metodológica del proyecto. Durante este periodo, se ha llevado a cabo una investigación y un estudio en profundidad sobre los principios operativos y las especificaciones técnicas de las aplicaciones que se van a usar y de las tecnologías que se van a implementar a lo largo del desarrollo.

Fase 2: Creación de datasets y preparación de datos

Al ser un proyecto de aprendizaje profundo, esta fase representa una de las mayores cargas de trabajo debido a su empeño en la búsqueda y recopilación de imágenes y etiquetado manual de esas mismas.

Fase 3: Programación y desarrollo del modelo

Esta fase abarca el núcleo del desarrollo técnico y de programación del proyecto. Se implementan de forma paralela los dos enfoques del proyecto: por un lado, la metodología que combina el YOLO y Keras; por el otro, la metodología directa basada en un modelo YOLO multiclase. Además, se desarrolla el script de conexión segura con la API de Google Gemini para la generación del diagnóstico y las recomendaciones de un tratamiento sobre la patología en concreto.

Fase 4: Evaluación y selección del modelo

Una vez entrenados los modelos, se procede a su validación estadística y análisis. Aquí simplemente se seleccionará en base a las distintas pruebas obtenidas los mejores modelos.

Fase 5: Documentación y entrega final

Esta fase final se desarrolla de manera transversal al cierre del proyecto. Comprende la redacción de la memoria final del Trabajo de Fin de Grado. Incluye también el proceso de preparación del repositorio de código final y la maquetación de los recursos necesarios para la defensa pública del sistema.

Capítulo 4

Análisis

4.1. Representación Digital de la Imagen

En el ámbito de la visión por computadora, las imágenes digitales se conceptualizan como estructuras matemáticas discretas. Una imagen en color se define formalmente como una matriz tridimensional o tensor de dimensiones $H \times W \times C$, donde H representa la altura (*height*), W la anchura (*width*) y C el número de canales de color. Cada elemento de esta matriz, denominado píxel, almacena un valor numérico entero que cuantifica la intensidad luminosa en una coordenada espacial específica. [25]

En esta figura 4.1 se muestra una representación visual de lo anterior (imagen sacada de [26]).

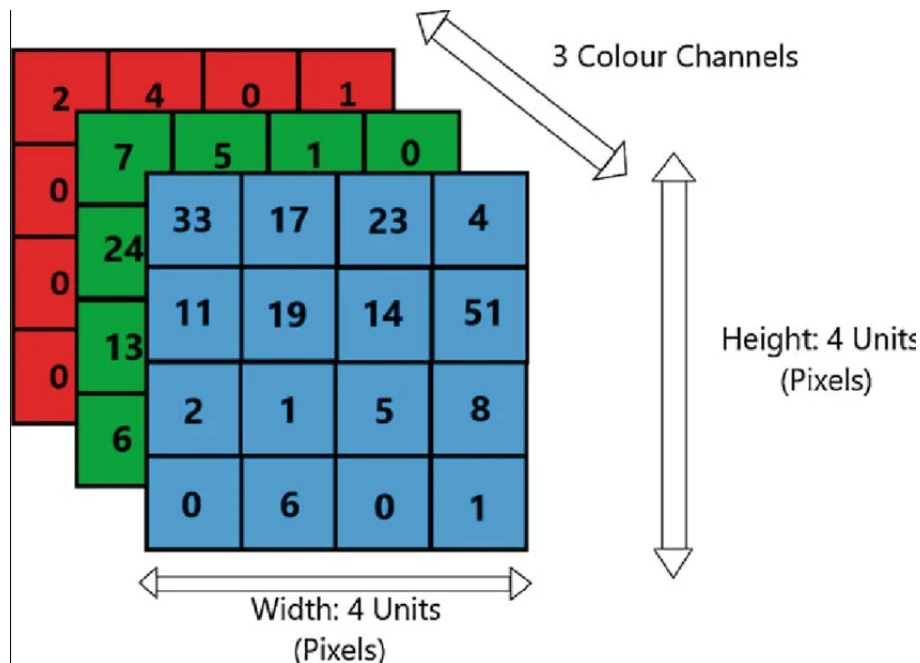


Figura 4.1: IMAGEN VISUAL $H \times W \times C$

4.1.1. Espacios de Color y Preprocesamiento

La interpretación de estas características depende del espacio de color seleccionado. Las transformaciones y el análisis de texturas en superficies vegetales se rigen por los siguientes estándares [27]:

- **Espacio RGB (Red, Green, Blue):** Es el modelo aditivo estándar donde cada canal representa la intensidad de las componentes roja, verde y azul en un rango discreto de $[0, 255]$ (para codificaciones de 8 bits). Constituye la representación nativa de captura en los sensores ópticos cotidianos.
- **Normalización y Escala:** Para garantizar la estabilidad numérica durante la optimización de redes neuronales, es una práctica mandatoria transformar los valores absolutos de los píxeles a un rango continuo $[0, 1]$ mediante la división matricial por el factor 255,0. Esta operación elimina sesgos de iluminación y acelera la convergencia del gradiente.
- **Restricciones Dimensionales:** Los pipelines de procesamiento imponen dimensiones fijas para unificar el tamaño de los tensores de entrada. En este proyecto, las muestras son sometidas a un reescalado forzoso a resoluciones estructuradas (512×512 píxeles para la arquitectura convolucional y dimensiones compatibles para el detector de objetos), preservando la consistencia topológica de las matrices. Cabe aclarar que en la cita anterior [27] en vez de 512×512 píxeles es de 256×256 píxeles.

Figura 4.2 (Fuente: [28]).



Figura 4.2: IMAGEN VISUAL ESPACIOS DE COLOR

4.2. Técnicas de Aumento de Datos (Data Augmentation)

El éxito de los modelos basados en aprendizaje profundo está estrictamente condicionado por el volumen y la variabilidad del conjunto de datos de entrenamiento. En entornos agrícolas reales, los factores ambientales introducen distorsiones ópticas que pueden inducir al sobreentrenamiento (*overfitting*) si el modelo memoriza el ruido del entorno.

Para mitigar este riesgo, se emplean técnicas de aumento de datos analíticas a través de la librería Albumentations, la cual manipula las matrices de píxeles sin alterar la etiqueta de la clase. Estas transformaciones se dividen en dos categorías esenciales:

Figura 4.3 (Fuente: [29]).



Figura 4.3: IMAGEN SOBRE UN EJEMPLO DE DISTINTAS TONALIDADES DE COLOR APLICADAS PARA AUMENTO DE DATOS

4.2.1. Transformaciones Espaciales y Geométricas

Modifican la posición y orientación de las estructuras dentro del plano de la imagen, forzando al modelo a aprender características invariantes a la perspectiva:

- **Inversiones Esenciales (*Flips*):** Rotaciones especulares horizontales y verticales que simulan diferentes orientaciones del fruto en el árbol o en las líneas de transporte.

- **Rotaciones y Recortes Aleatorios (*Cropping*):** Extracción de submatrices de la imagen original para entrenar la red en la detección de patologías a diferentes distancias y escalas focales.

Figura 4.4 (Fuente: [30]).

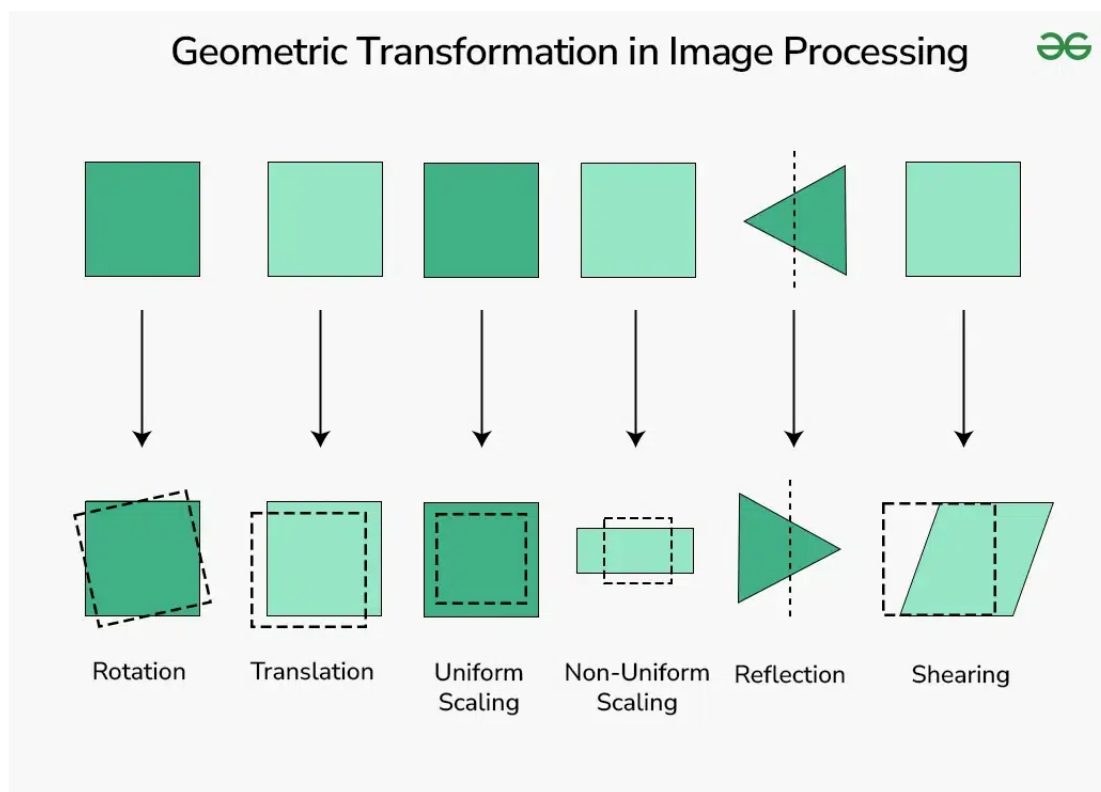


Figura 4.4: DISTINTOS TIPOS DE TRANSFORMACIONES ESPACIALES Y GEOMÉTRICAS

4.2.2. Transformaciones Fotométricas y del Pixel

Alteran los valores de los canales de color y la nitidez, simulando variaciones en las condiciones atmosféricas e imperfecciones del sensor óptico:

- **Modificaciones de Brillo y Contraste:** Ajustes no lineales sobre las matrices de intensidad para emular la incidencia de luz solar directa o sombras densas en el campo.
- **Desenfoque Gaussiano (*Gaussian Blur*):** Aplicación de un filtro de suavizado matemático que emula pérdidas de enfoque causadas por el movimiento de la cámara o vibraciones de la maquinaria de inspección.

Figura 4.5 (Fuente: [31]).



Figura 4.5: IMAGEN SOBRE MODIFICACIONES DE BRILLO, CONTRASTE Y DES-ENFOQUE GAUSSIANO

4.3. Dinámica del Aprendizaje y Convergencia Semántica

El entrenamiento de una arquitectura de visión artificial se rige por un ciclo iterativo de optimización dividido en dos fases fundamentales, cuyo objetivo es ajustar dinámicamente los valores de los *kernels* y los pesos sinápticos [32]:

- **Propagación hacia adelante (*Forward Pass*):** Las matrices de la imagen fluyen a través de los bloques de convolución, activación y submuestreo hasta ser aplanadas en un vector unidimensional que alimenta las capas densas. La salida predictiva resultante es evaluada frente a la etiqueta real empleando funciones de pérdida como la entropía cruzada.
- **Propagación hacia atrás (*Backward Pass*):** Basándose en el cálculo matemático del gradiente de la función de pérdida, el algoritmo de retropropagación distribuye el error en sentido inverso a través de la red. Empleando optimizadores estocásticos, los pesos de los filtros se actualizan de manera infinitesimal para minimizar la pérdida en la siguiente iteración.

Figura 4.6 (Fuente: [33]).

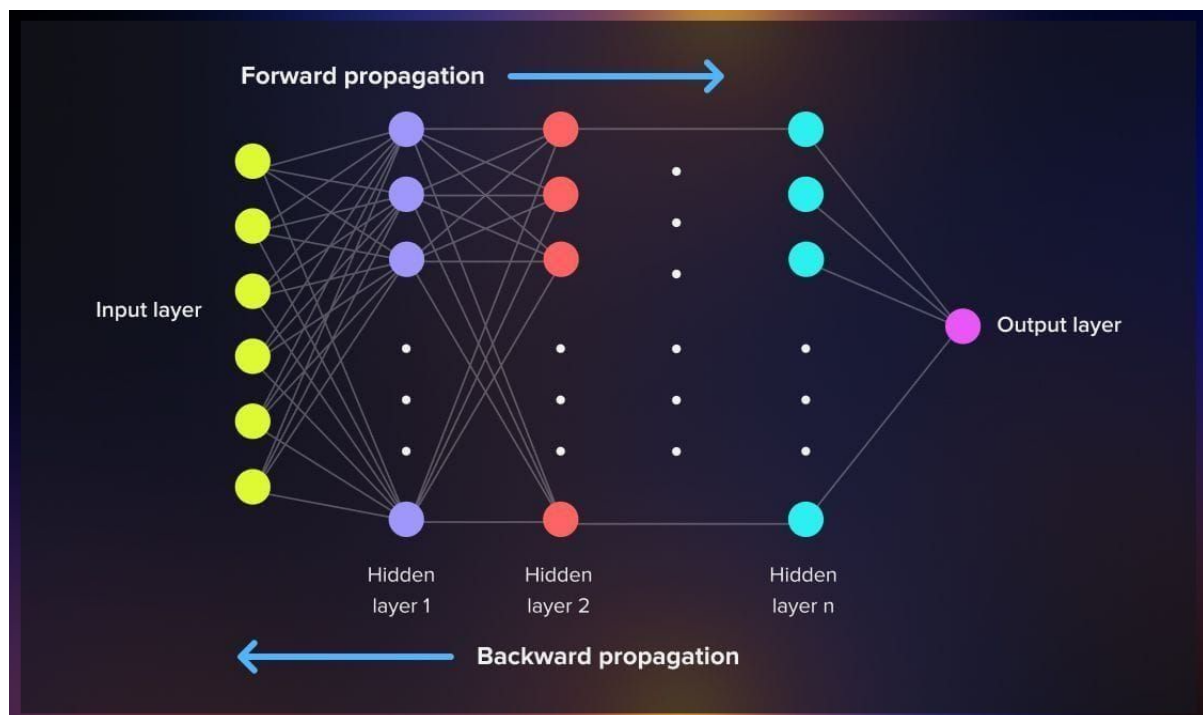


Figura 4.6: PROPAGACIÓN FORWARD Y BACKWARD

Capítulo 5

Diseño

5.1. El paradigma del Aprendizaje Automático

El Aprendizaje Automático (*Machine Learning*) constituye un pilar fundamental de las ciencias de la computación cuyo propósito es dotar a los sistemas de la capacidad de asimilar conocimiento implícito a partir de la experiencia empírica. En lugar de codificar algoritmos basados en reglas lógicas rígidas y programación explícita, este enfoque busca que las máquinas infieran patrones abstractos latentes en los datos para proyectar predicciones precisas ante escenarios totalmente desconocidos. Mediante la optimización matemática de sus componentes, es factible estructurar modelos cuyo rendimiento iguala o supera los umbrales de precisión humana en tareas especializadas [34].

5.1.1. Tipologías de Aprendizaje

Dependiendo de la naturaleza de los datos de entrada y la interacción con el entorno, el aprendizaje se clasifica en dos vertientes principales:

- **Aprendizaje No Supervisado:** Orientado al descubrimiento de estructuras intrínsecas y relaciones ocultas dentro de conjuntos de datos carentes de etiquetas o respuestas predefinidas. Sus aplicaciones clave abarcan el agrupamiento geométrico (*clustering*) mediante algoritmos como k -medias o DBSCAN, y la reducción de dimensionalidad (como PCA y t -SNE), esenciales para condensar topologías de alta fidelidad en representaciones visuales bidimensionales.
- **Aprendizaje Supervisado:** Metodología que se rige por entrenar al modelo con un conjunto de datos previamente etiquetado donde el objetivo (*ground truth*) se encuentra perfectamente definido. El sistema modela una función de mapeo capaz de clasificar elementos en categorías discretas (como la identificación de patologías foliares) o resolver problemas de regresión numérica continua [35].

Figura 5.1 (Fuente: [36]).

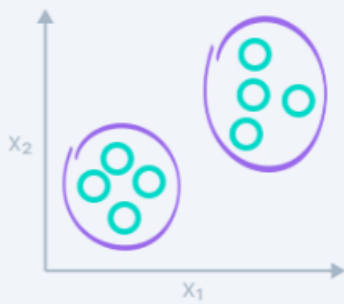
Supervised learning	Unsupervised learning
Input data is labeled	Input data is unlabeled
Has a feedback mechanism	Has no feedback mechanism
Data is classified based on the training dataset	Assigns properties of given data to classify it
Divided into Regression & Classification	Divided into Clustering & Association
Used for prediction	Used for analysis
Algorithms include: decision trees, logistic regressions, support vector machine	Algorithms include: k-means clustering, hierarchical clustering, apriori algorithm
A known number of classes	A unknown number of classes
	

Figura 5.1: DIFERENCIAS ENTRE LAS DOS ARQUITECTURAS

5.2. Fundamentos de la Neurona Artificial y el Perceptrón

La génesis del aprendizaje profundo se remonta al modelo matemático propuesto por McCulloch y Pitts en 1943, el cual buscaba replicar de forma simplificada el comportamiento bioeléctrico del sistema nervioso central [37]. Posteriormente, en 1957, Frank Rosenblatt materializó estos conceptos en el Perceptrón, la primera unidad de cómputo supervisado capaz de resolver problemas linealmente separables [38].

El verdadero punto de inflexión de la disciplina ocurrió con el desarrollo del Perceptrón Multicapa (MLP) y la introducción del algoritmo de retropropagación (*backpropagation*), rompiendo las limitaciones operacionales del modelo primitivo y permitiendo el procesamiento de fronteras de decisión no lineales complejas [39].

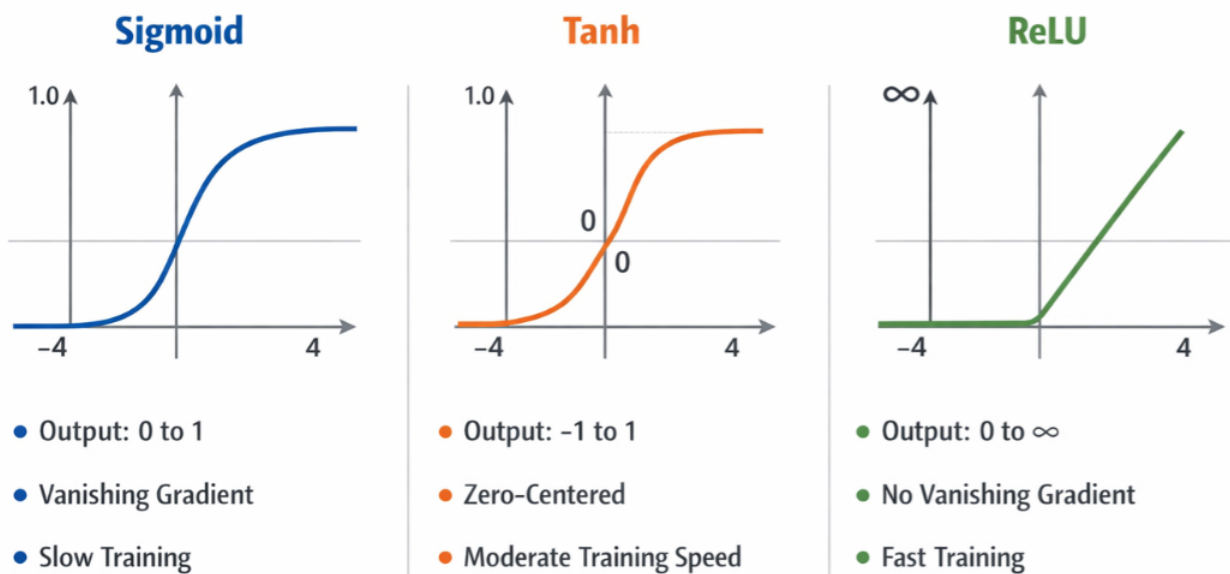
5.2.1. Estructura computacional de la Neurona

Una neurona artificial actúa como un nodo de procesamiento que transforma un vector de entradas mediante la siguiente arquitectura paramétrica:

- **Pesos Sinápticos (w):** Coeficientes numéricos que cuantifican la fuerza de conexión e importancia de cada característica de entrada; representan el núcleo del conocimiento almacenado por la red.
- **Sesgo (*Bias*, b):** Parámetro aditivo independiente que desplaza la función de activación, otorgando un grado de libertad crucial para ajustar el modelo con mayor refinamiento.
- **Función de Activación (ϕ):** Operación matemática que introduce propiedades no lineales en el sistema, permitiendo modelar espacios multidimensionales complejos. Tradicionalmente se empleaban funciones sigmoideas, pero la industria ha adoptado la Unidad Rectificada Lineal (ReLU) debido a su simplicidad computacional, la mitigación del desvanecimiento del gradiente y el fomento de activaciones dispersas [40].
- **Función de Pérdida (*Loss Function*):** Métrica que evalúa cuantitativamente la desviación entre la predicción de la red y el valor real, guiando la optimización matemática de los parámetros durante el entrenamiento.

Sigmoid vs Tanh vs ReLU:

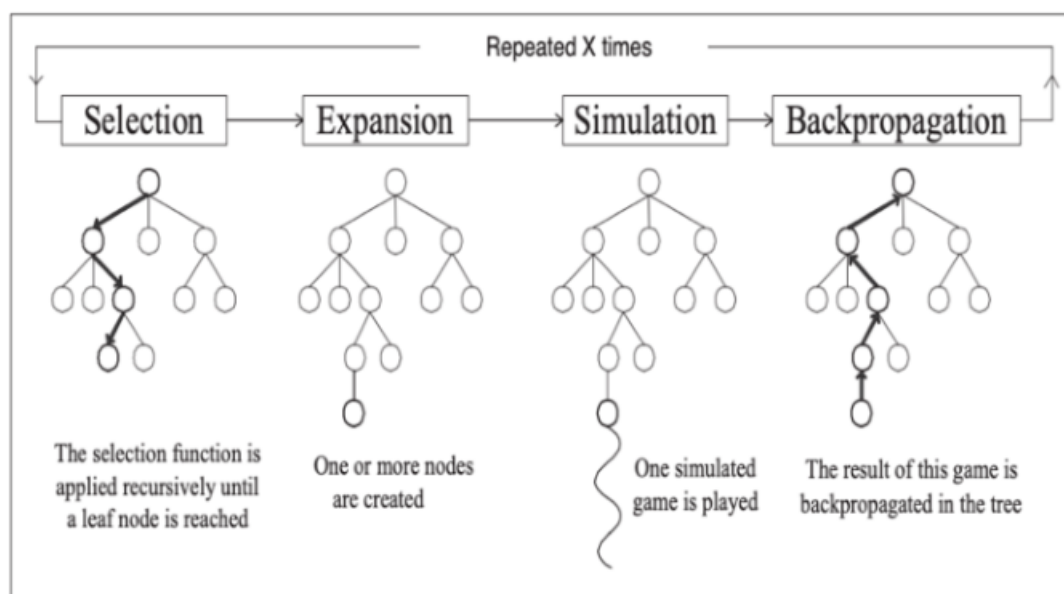
Activation Function Comparison



5.3. Evolución y taxonomía de las Redes Neuronales

El crecimiento exponencial en la capacidad de cómputo de hardware especializado (como GPUs y TPUs) ha propiciado la diversificación de arquitecturas neuronales orientadas a dominios de datos específicos [41]:

- **Redes Neuronales Artificiales (ANN / MLP):** Estructuras totalmente conectadas (*fully connected*) donde cada nodo se enlaza con la totalidad de la capa subsecuente. Presentan un alto rendimiento en datos tabulares y relaciones generalistas no lineales [42].
- **Redes Neuronales Recurrentes (RNN):** Modelos diseñados con conexiones cíclicas retroalimentadas, idóneos para el análisis de secuencias lógicas y series temporales al retener memoria latente de estados pasados.
- **Redes Generativas Antagónicas (GAN):** Arquitectura dual compuesta por un modelo generador y uno discriminador que compiten entre sí, especializada en la síntesis y mejora de datos e imágenes realistas.
- **Redes Neuronales Convolucionales (CNN):** Algoritmos de aprendizaje profundo diseñados específicamente para el procesamiento de matrices bidimensionales y tensores de imágenes, emulando la organización jerárquica del córtex visual biológico.



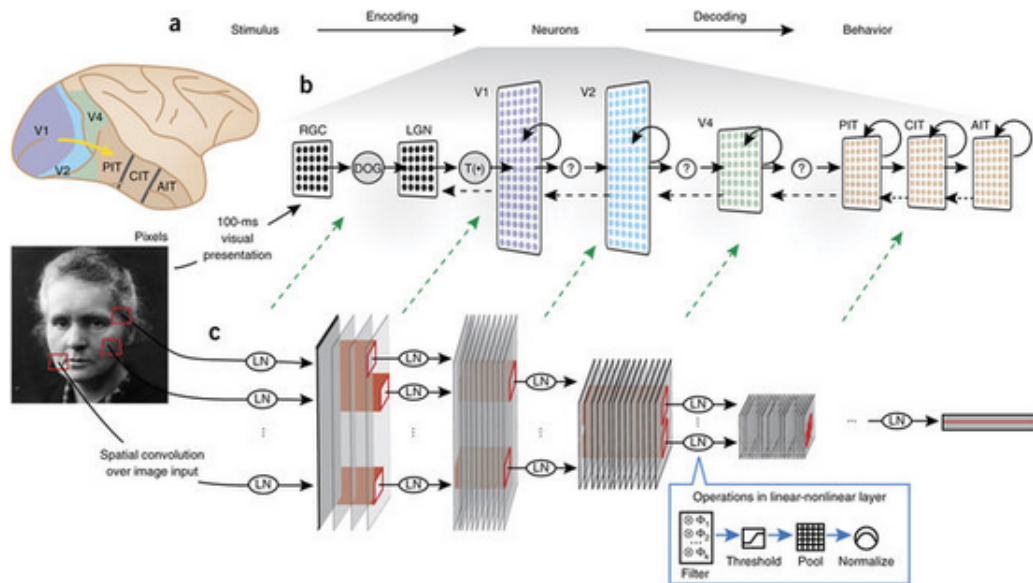
5.4. Inspiración biológica de las Arquitecturas Convolucionales

El diseño arquitectónico de las CNN encuentra su fundamento científico en los descubrimientos neurofisiológicos de los doctores Hubel y Wiesel en las décadas de 1950 y

1960 [43, 44]. Mediante el análisis de las respuestas neuronales en el córtex visual animal ante estímulos ópticos, determinaron que el procesamiento de imágenes no ocurre de forma global, sino a través de una descomposición jerárquica y local de la información. Su trabajo, galardonado con el Premio Nobel en 1981, sentó las bases para estructurar el procesamiento artificial en cascada [45].

5.4.1. Mapeo Funcional del córtex visual en la CNN

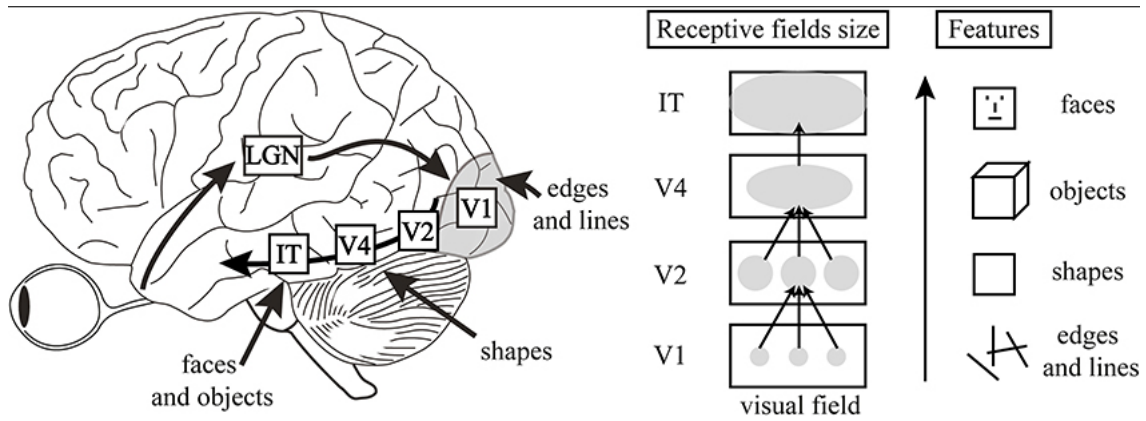
El cerebro procesa la información visual dividiendo la tarea entre áreas especializadas de la corteza, una topología emulada de forma análoga por las capas de una CNN [46, 47]:



- **Áreas V1 y V2 (Corteza Primaria y Secundaria):** Encargadas de registrar la orientación espacial, bordes primitivos, frecuencias y tonalidades cromáticas iniciales. Equivalen directamente a los primeros bloques de extracción de características y capas convolucionales iniciales de una red.
- **Áreas V3 y V4:** Procesan formas geométricas intermedias, texturas complejas y la constancia del contorno del objeto. Se corresponden con las capas convolucionales intermedias del modelo.
- **Áreas V5 y V6 (Procesamiento de Alto Nivel y Entorno):** Dedicadas al reconocimiento semántico global del objeto, su movimiento e integración contextual. En la arquitectura artificial, esta lógica se implementa en las capas densas totalmente conectadas y los módulos de clasificación final.

5.5. Extracción de características mediante convolución

A diferencia de los enfoques de aprendizaje automático clásico que requerían la extracción manual de descriptores visuales, las Redes Neuronales Convolucionales (CNN) automatizan este proceso mediante el aprendizaje jerárquico de características.



5.5.1. La operación de convolución

Matemáticamente, una capa convolucional aplica un conjunto de filtros o núcleos (*kernels*) sobre la matriz de la imagen de entrada. Un *kernel* es una matriz de dimensiones reducidas (típicamente 3×3 o 5×5) que se desplaza a lo largo del tensor de entrada realizando un producto escalar elemental acumulado. La operación discreta de convolución bidimensional para una posición espacial se define formalmente como:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n) \quad (5.1)$$

Donde I representa la matriz de la imagen, K denota el *kernel* convolucional y $S(i, j)$ es el mapa de características resultante (*feature map*). Las primeras capas de la red capturan patrones visuales de bajo nivel (bordes, contornos y texturas de la corteza de la naranja), mientras que las capas profundas combinan estos mapas para reconocer estructuras complejas de alto nivel.

Figura 5.2 (Fuente: [48]).

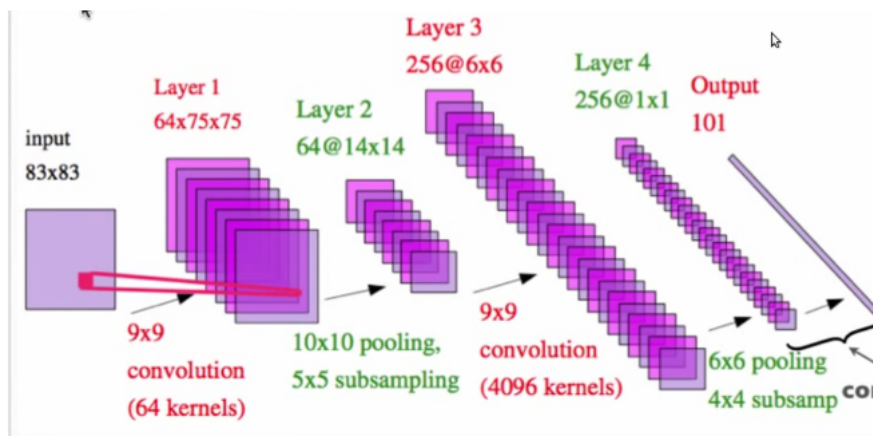


Figura 5.2: OPERACIÓN CONVOLUCIÓN

5.5.2. Reducción de dimensionalidad (*Pooling*)

Para reducir la carga computacional y conferir invariancia a pequeñas translaciones, se intercalan capas de submuestreo o *Pooling*. La técnica más extendida es el *Max-Pooling*, la

cual extrae exclusivamente el valor máximo dentro de una ventana deslizante, preservando las activaciones más prominentes del mapa de características.

Figura 5.3 (Fuente: [49]).

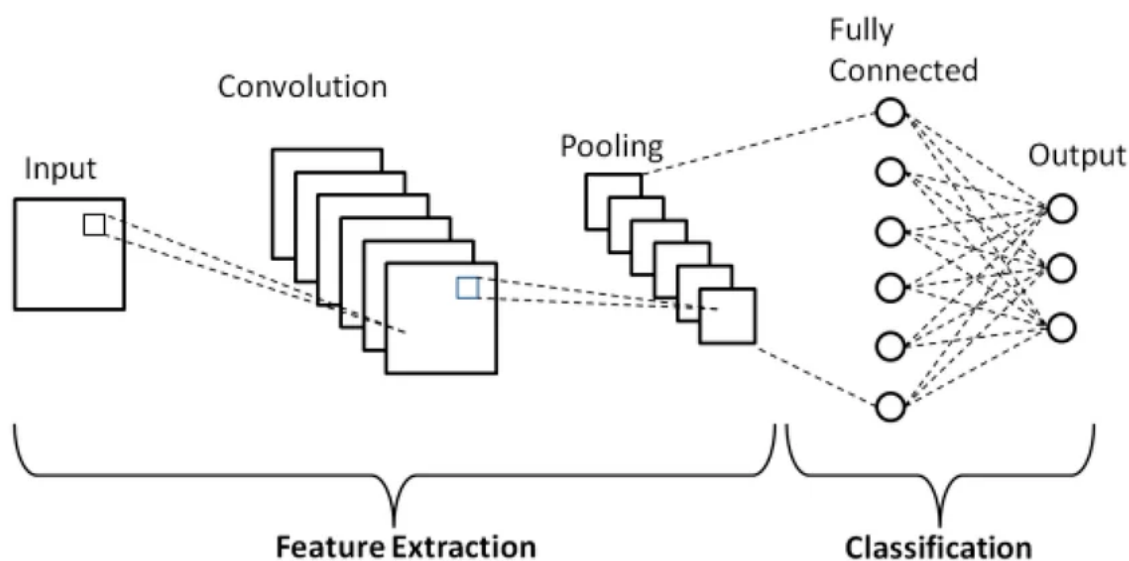


Figura 5.3: RED NEURONAL CONVOLUCIONAL ESQUEMA

5.6. El nuevo paradigma: Grandes Modelos de Lenguaje (LLMs)

En la última década, la taxonomía de las redes neuronales ha experimentado una disrupción profunda con la emergencia de los Grandes Modelos de Lenguaje (*Large Language Models* o LLMs). Estas arquitecturas abandonan la estructura cíclica de las RNN tradicionales en favor del mecanismo de autoatención (*self-attention*), introducido por la arquitectura Transformer [50].

A diferencia de las CNN, que procesan la información aplicando filtros locales y jerárquicos sobre matrices espaciales, los LLMs computan relaciones geométricas globales entre todos los elementos de una secuencia (tokens) de forma simultánea. Mediante el preentrenamiento auto-supervisado a escala masiva sobre corpus de texto multilingües, el modelo no solo aprende la sintaxis del lenguaje, sino que emerge una comprensión semántica abstracta y capacidades de razonamiento conceptual (*in-context learning*) [51].

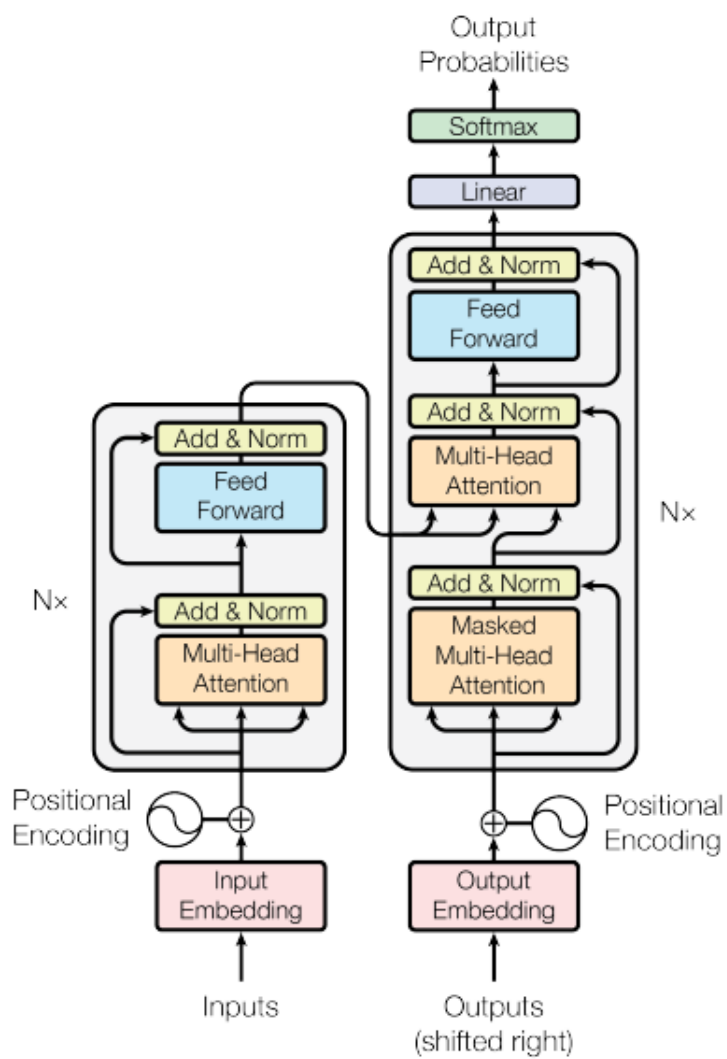


Figura 5.4: ARQUITECTURA BASE TRANSFORMER Y MECANISMO DE ATENCIÓN

5.6.1. Operación matemática fundamental: Autoatención Escalada

A diferencia de las capas convolucionales que confinan su operación a vecindades espaciales locales mediante un *kernel*, un LLM evalúa de forma dinámica e instantánea la relación semántica de cada palabra (token) con todas las demás dentro de una secuencia, sin importar la distancia física que las separe.

Matemáticamente, este proceso se modela mediante el mecanismo de **Atención de Producto Escalar Escalado** (*Scaled Dot-Product Attention*). El vector de entrada transformado de cada token se proyecta linealmente en tres matrices fundamentales de pesos entrenables: Consultas (Q), Claves (K) y Valores (V). La operación que determina la distribución de pesos e importancia relativa entre los elementos se define formalmente como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (5.2)$$

Donde Q , K y V representan las matrices de consultas, claves y valores respectivamente, K^T denota la transpuesta de la matriz de claves, y el término $\sqrt{d_k}$ actúa como un factor de escalado basado en la dimensionalidad de las claves para evitar que los gradientes de la función *softmax* se desvanezcan durante el entrenamiento. Esta operación permite que el modelo genere representaciones vectoriales dinámicas en función del contexto exacto de la frase, posibilitando capacidades emergentes de razonamiento abstracto y generalización conceptual [51].

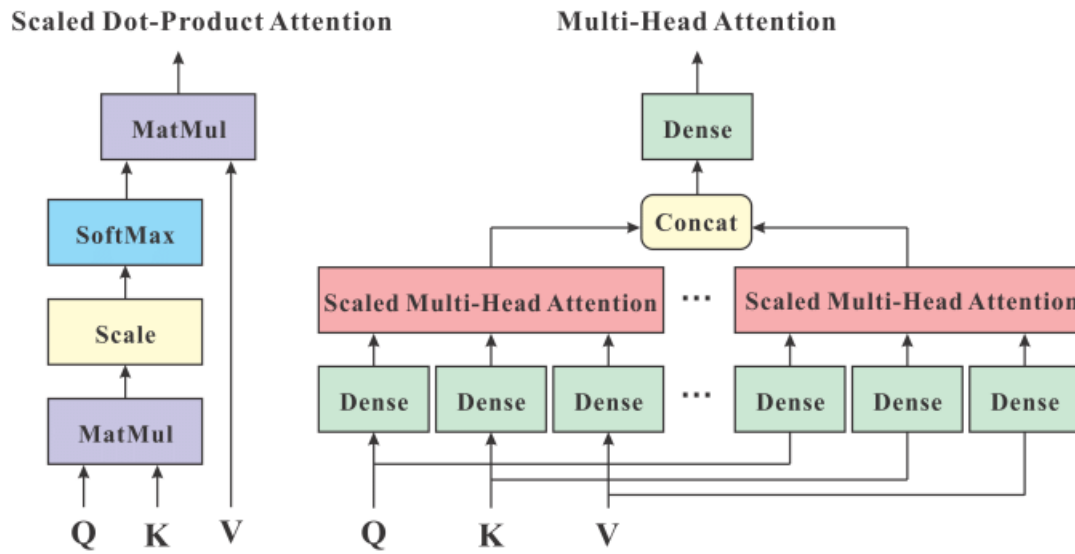


Figura 5.5: ATENCIÓN DE PRODUCTO ESCALADO

5.6.2. Más LLM: ChatGPT y Gemini

La consolidación de la arquitectura Transformer permitió la transición desde modelos fundacionales teóricos hacia sistemas de producción masiva capaces de interactuar mediante lenguaje natural estructurado. El exponente que democratizó esta tecnología fue la familia GPT (*Generative Pre-trained Transformer*) de OpenAI, cuya iteración ChatGPT introdujo el Aprendizaje por Refuerzo a partir de la Retroalimentación Humana (RLHF). Este método de alineación actúa como una capa de optimización posterior al entrenamiento matemático básico, forzando al modelo a refinar sus respuestas para que no solo sean probabilísticamente coherentes, sino también seguras y alineadas con las instrucciones del usuario (*instruction tuning*) [52].

Por otro lado, Google expandió los límites de estas arquitecturas mediante el desarrollo de Gemini, una familia de modelos nativamente multimodales. A diferencia de los LLMs tradicionales que requerían codificadores externos para interpretar estímulos visuales o auditivos, Gemini unifica el procesamiento de texto, código, audio e imágenes dentro del mismo espacio latente desde su fase de preentrenamiento. Esto permite una transferencia de conocimiento cruzada que emula de forma más cercana la integración contextual que ocurre en las áreas de procesamiento de alto nivel del cerebro humano.

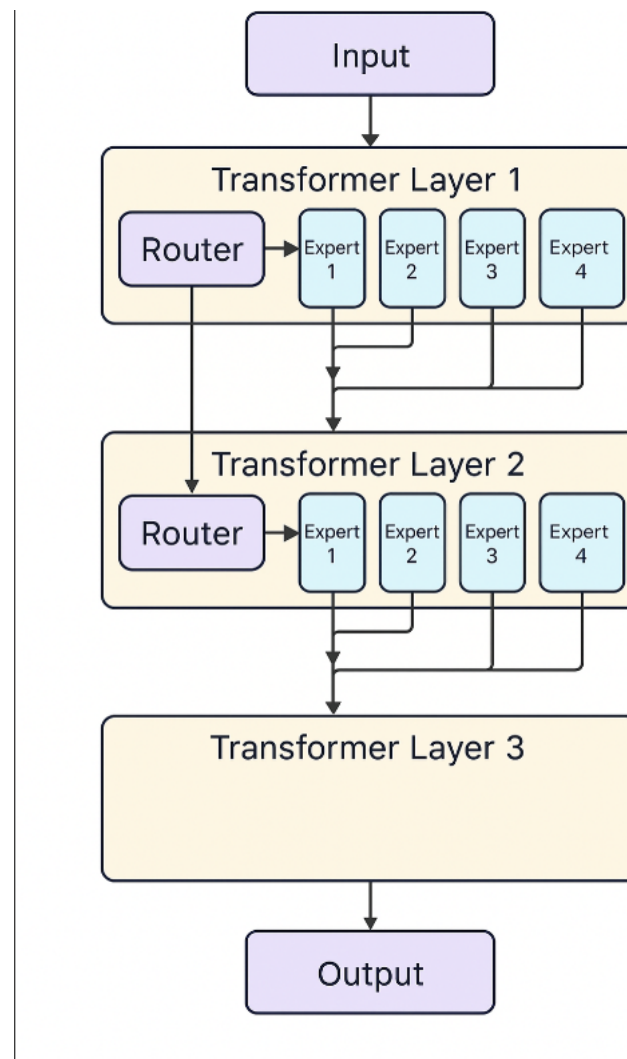


Figura 5.6: ARQUITECTURA DE MEZCLA DE EXPERTOS (MoE)

Capítulo 6

Implementación y pruebas

Este capítulo aborda de manera detallada las fases de desarrollo técnico y la posterior validación del sistema.

6.1. Implementación

6.1.1. Análisis de Datos

Roboflow constituye una de las plataformas y repositorios más relevantes para la gestión, preprocesamiento y optimización de conjuntos de datos orientados a la visión artificial. El ecosistema proporciona un conjunto de datos considerablemente limpio y estructurado.

Esta plataforma se considera como un estándar en la industria y la academia, sin embargo, los argumentos en contra señalan que la corta variabilidad de los fondos y las condiciones de iluminación controladas no representan fielmente los escenarios caóticos de la vida cotidiana. Además, se critica que las etiquetas y cajas de delimitación (*bounding boxes*) definidas suelen pecar de una excesiva amplitud o ambigüedad semántica. Por ejemplo, una categoría general como “vehículo” en estos conjuntos de datos puede agrupar indiscriminadamente desde automóviles compactos hasta camiones de carga pesada, los cuales poseen características morfológicas y escalas espaciales radicalmente opuestas.

6.1.2. Configuración y Estructuración de Datasets en Roboflow

La fase de Roboflow se centró en el etiquetado y la delimitación espacial (*labeling*) de naranjas para entrenar dos modelos de detección basados en la arquitectura YOLO. El propósito de este enfoque es la comparación entre dos metodologías distintas y ver cual de las dos es más eficiente y da mejores resultados:

- **Primer Dataset (Clasificación de Estado General):** Orientado al primer modelo YOLO, cuyo objetivo es realizar una discriminación binaria primaria entre cítricos en *buen estado* o *mal estado*. Este conjunto consta de un volumen de 1,861 imágenes que contienen un total de 5,558 instancias anotadas (2,784 sanas y 2,774 con patologías). La discrepancia entre el número total de imágenes y el volumen de

etiquetas se debe a múltiples naranjas dentro de un mismo campo visual. [Enlace al primer dataset.](#)

- **Segundo Dataset (Identificación Específica de Patologías):** Dirigido al segundo modelo YOLO, diseñado para realizar un diagnóstico granular sobre los frutos afectados. Este dataset comprende 906 imágenes y se enfoca en discriminar entre frutos completamente sanos y siete enfermedades específicas: *canker* (cancrosis), *greening*, manchas negras, moho verde, *scab*, trips y una categoría genérica para otros defectos en las naranjas. [Enlace al segundo dataset.](#)

Para garantizar la reproducibilidad del experimento, la distribución detallada de las anotaciones de ambos conjuntos de datos se sintetiza en la Tabla 6.1.

Cuadro 6.1: Distribución de imágenes etiquetadas en los datasets de Roboflow.

Dataset	Total Imágenes	Clasificación de Naranjas Etiquetadas
Dataset 1 (General)	1,861	Buen estado: 2,784 Mal estado: 2,774
Dataset 2 (Patologías)	906	Sana: 767 Canker: 227 Scab: 204 Greening: 167 Moho Verde: 160 Manchas Negras: 142 Trips: 136 Otros defectos: 127

La asimetría en el volumen de datos y el grado de balanceo entre ambos conjuntos responde a una estrategia metodológica orientada a optimizar el pipeline de detección. El primer dataset cuenta con un mayor número de muestras y una distribución de clases estrictamente homogénea debido a la necesidad de maximizar el recall del primer modelo YOLO; asegurar la robustez de esta primera etapa de discriminación binaria es crítico, puesto que servirá como base de comparación para evaluar el rendimiento entre la arquitectura implementada en Keras y el segundo modelo YOLO de la segunda metodología.

Por otro lado, la menor dimensión del segundo dataset se atribuye tanto a limitaciones técnicas y a restricciones temporales como se explicó en los costes del proyecto. En primer lugar, existía una menor disponibilidad de imágenes que presentaran manifestaciones visuales nítidas de las patologías específicas. En segundo lugar, el elevado coste temporal invertido en la fase de anotación y refinamiento del primer modelo YOLO obligó a optimizar los recursos disponibles, condensando el proceso de etiquetado de la segunda etapa sin comprometer la viabilidad del diagnóstico.

6.1.3. Preparación de los Datos antes del entrenamiento

Tal como se ilustra en la Figura 6.1, la distribución final del conjunto de datos se estructuró mediante una partición clásica, destinando un 70 % de las muestras al subconjunto de entrenamiento (*train*), un 20 % a pruebas (*test*) y el 10 % restante a la fase de validación (*validation*).

The screenshot shows the Roboflow dataset management interface for a dataset named "2800 buenas 2800 malas". The interface is divided into several sections:

- Versions:** A list of dataset versions on the left sidebar. The selected version is v9, which has 1861 images and a resolution of 850x850. Other versions include v8 (1151 images, 640x640), v7 (1040 images, 640x640), v6 (1080 images, 640x640), v5 (1217 images, 640x640), v4 (1160 images, 640x640), v3 (1048 images, 640x640), v2 (742 images, 640x640), and v1 (464 images, 640x640).
- Dataset Overview:** The main area shows the dataset name "2800 buenas 2800 malas", generated on Apr 9, 2026 by Jacobo. It includes buttons for "Download Dataset" and "Edit".
- Model Training:** A message states "This version doesn't have a model." and provides instructions on how to train a model using Roboflow Credits or upload custom weights. A "Train Model" button is present.
- Dataset Split:** A section showing the distribution of images. The "TRAIN SET" contains 1861 Images (100%), while the "VALID SET" and "TEST SET" both contain 0 Images.
- Preprocessing:** A section showing the applied preprocessing steps: "Auto-Orient: Applied" and "Resize: Stretch to 850x850".
- Augmentations:** A section showing that "No augmentations were applied".
- Image Gallery:** A row of 10 sample images showing various fruits, primarily oranges and lemons, with some showing signs of decay or damage.

Figura 6.1: DATASET 1

Cabe destacar que, inicialmente, la plataforma Roboflow asignó de forma automática la totalidad de las imágenes al bloque de entrenamiento. Para subsanar esta anomalía y garantizar un protocolo de evaluación riguroso que prevenga el sobreajuste (*overfitting*), se procedió a una reestructuración manual. Mediante este proceso, se generaron de forma independiente las carpetas correspondientes a los bloques de test y validación, redistribuyendo las imágenes de manera equilibrada y controlada para cumplir estrictamente con los porcentajes previamente establecidos.

A continuación, si se muestra la correcta distribución en el segundo dataset en la Figura 6.2

Versions

800 imagenes YOLO 2_3_Spli...

v4 906 600x600 Stretch to Jacobo

800 imagenes YOLO 2_2

v3 824 600x600 Stretch to Jacobo

800 imagenes YOLO 2

v2 824 600x600 Stretch to Jacobo

2025-06-19 9:39am

v1 99 400x400 Stretch to Jacobo

v4 800 imagenes YOLO 2_3_Split_mejorado

Generated on May 8, 2026 by Jacobo

Download Dataset Edit

This version doesn't have a model.

Train an optimized, state of the art model with Roboflow or upload a custom trained model to use features like Label Assist and Model Evaluation and deployment options like our auto-scaling API and edge device support.

Train Model

Uses Roboflow Credits View usage

How to Upload Custom Weights

906 Total Images

View All Images →



Dataset Split

TRAIN SET 70% 634 Images

VALID SET 10% 91 Images

TEST SET 20% 181 Images

Preprocessing

Auto-Orient: Applied
Resize: Stretch to 600x600

Augmentations

No augmentations were applied

Figura 6.2: DATASET 2

Para el Keras he usado un dataset a parte creado en distintas carpetas para clasificar las enfermedades correspondientes, aunque el flujo de inicialización de datos en el script implementa un fraccionamiento teórico del 80 % para el entrenamiento (*train_ds*) y un 20 % para la validación interna (*val_ds*), este último subconjunto fue omitido deliberadamente durante la fase de optimización iterativa (*model.fit*). En su lugar, el modelo fue expuesto a un régimen de aprendizaje basado puramente en el conjunto de entrenamiento. Para evaluar de manera estricta la capacidad de generalización del clasificador ante datos no vistos, las métricas de rendimiento y la matriz de confusión se calcularon utilizando un conjunto de datos independiente y personalizado, el cual actúa metodológicamente como un **Dataset de Evaluación Externa**.

A diferencia de como hemos implementado en YOLO, donde la validación se ejecuta de forma síncrona sobre una partición del mismo conjunto de datos original, en este bloque del proyecto se optó por un enfoque de evaluación externo. Esta decisión estratégica responde a la necesidad de testear el modelo otro método de evaluación distinto.

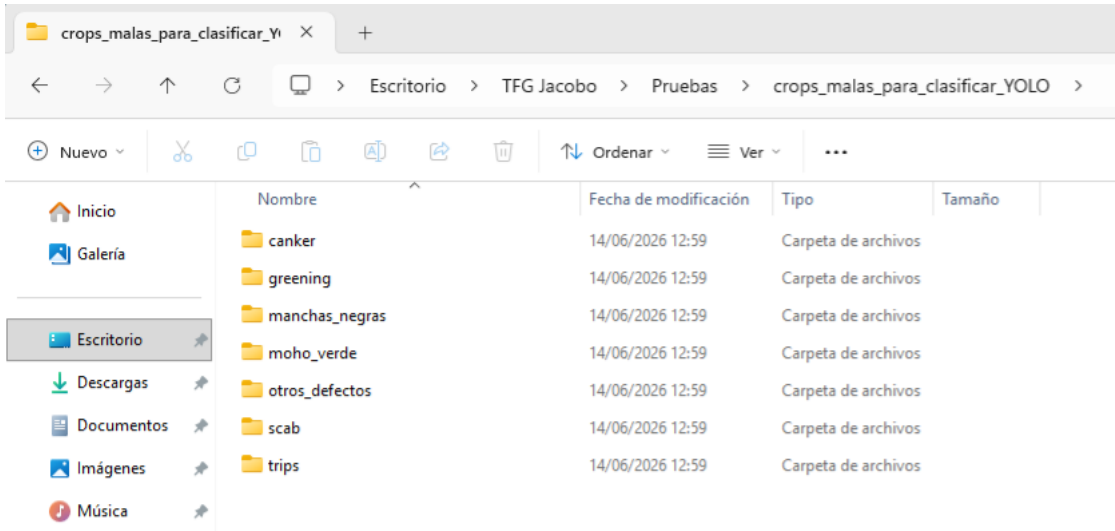


Figura 6.3: DATASET KERAS

Cuadro 6.2: Distribución de imágenes etiquetadas en el dataset de Keras.

Dataset	Total Imágenes	Clasificación de Naranjas Etiquetadas
Dataset Keras (Patologías)	1617 (de una sola naranja)	Canker: 274 Scab: 161 Greening: 155 Moho Verde: 200 Manchas Negras: 166 Trips: 204 Otros defectos: 457

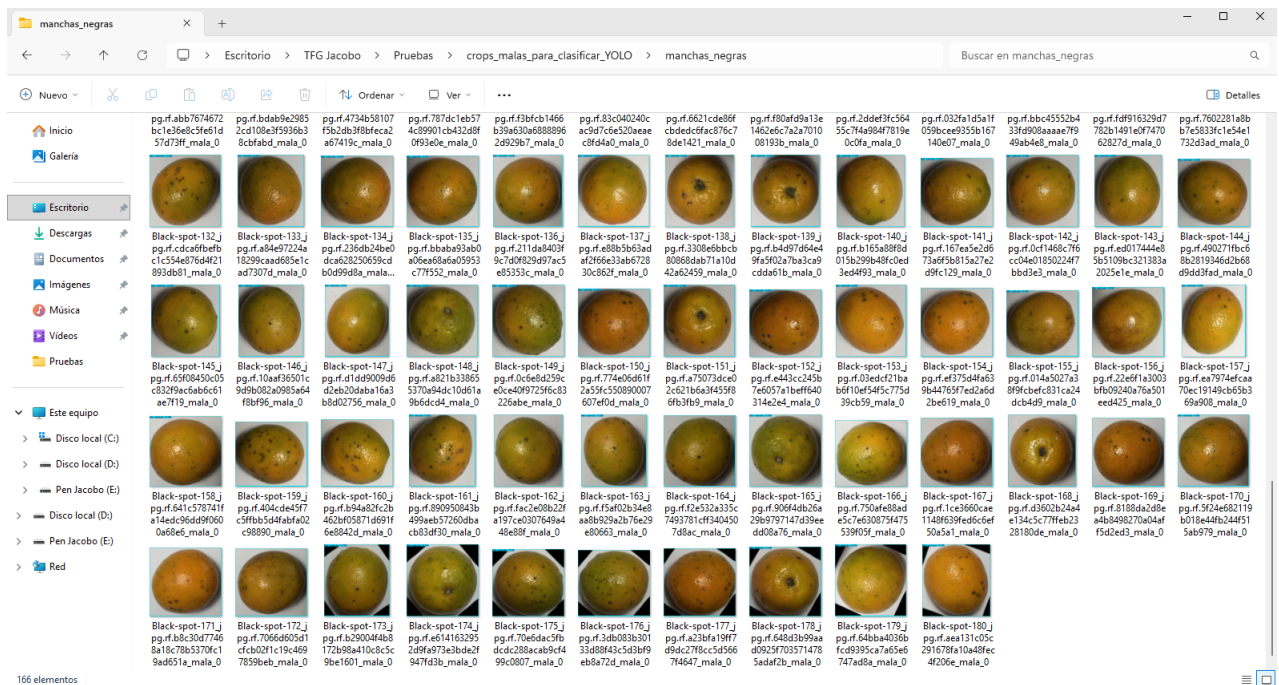


Figura 6.4: EJEMPLO DENTRO DE UNA CARPETA DEL DATASET DEL KERAS

6.1.4. Fase de entrenamiento de los modelos

Para la fase de entrenamiento de nuestros modelos se ha empleado la versión v11 de YOLO. La elección de esta versión frente a iteraciones alternativas se fundamenta en criterios de rendimiento y viabilidad temporal dentro del cronograma del proyecto:

- **Estabilidad y Rendimiento:** Durante la fase de investigación sobre la versión YOLOv12 no reportó incrementos significativos ni mejoras generalizadas en las métricas o eficiencia en el contexto específico de este dataset, justificando el retorno a una arquitectura consolidada y eficiente como la v11.
- **Restricciones Temporales:** Aunque el ecosistema de *Ultralytics* ha continuado evolucionando con el reciente lanzamiento de YOLO26, dicha versión fue introducida al mercado con posterioridad al inicio, diseño e implementación de la metodología de este estudio. Por tanto, se optó por mantener la versión v11.

El código de todo el proyecto se puede encontrar aquí: [Enlace de Github](#).

Primero se llevó a cabo la fase de entrenamiento de los modelos, durante esta etapa, se llevó a cabo una fase de ajuste y experimentación con diferentes hiperparámetros para evaluar el rendimiento y seleccionar la configuración que ofreciera los mejores resultados. Sin embargo, esto no fue suficiente para nuestro primer YOLO y se tuvo que hacer una fase de aumento de imágenes para mejorar las métricas.

Listado 6.1: Código para aumentar el número de imágenes de nuestra primera metodología

```
# =====
# CÓDIGO PARA REALIZAR ALGUNOS AUMENTOS DE DATOS (SOLAMENTE USADO EN EL PRIMER
# YOLO)
# PROYECTO: Estudio y aplicación de YOLO, redes convolucionales (CNN) y modelos
#           de lenguaje (LLM) para la detección, clasificación y tratamiento de
#           naranjas
# =====

import os
import cv2
import albumentations as A
import random

# === CONFIGURACIÓN DE RUTAS Y PARÁMETROS OPERATIVOS ===
# Directorio de entrada con las muestras fotográficas de cítricos originales
input_folder = "C:/Users/IVIA/Downloads/Citrus/Pruebas CITRUS 03-05-2018/
Data_augmentation"
# Directorio destino optimizado para el almacenamiento del set extendido
# expandido
output_folder = "C:/Users/IVIA/Downloads/Data augmentation imagenes"
os.makedirs(output_folder, exist_ok=True)

# Factor multiplicador de expansión (Número de variantes sintéticas a generar
# por imagen)
num_augmented = 5

# DEFINICIÓN DEL PIPELINE DE TRANSFORMACIONES ESTOCÁSTICAS
```

```

# Configuración geométrica y pixelar para robustecer la invariancia del modelo
  CNN/YOLO
transform = A.Compose([
    A.RandomRotate90(),          # Rotaciones ortogonales aleatorias (0, 90,
    180, 270)
    A.HorizontalFlip(p=0.5),      # Inversión especular horizontal con
    probabilidad del 50%
    A.VerticalFlip(p=0.2),        # Inversión especular vertical con
    probabilidad del 20%
    A.RandomBrightnessContrast(p=0.5), # Perturbación estocástica de las curvas
    de brillo y contraste
    A.Blur(blur_limit=3, p=0.2),  # Simulación de desenfoque por movimiento o
    pérdida de foco óptico
    A.LongestMaxSize(max_size=640), # Redimensión proporcional basada en el eje
    de mayor longitud
    A.PadIfNeeded(min_height=640, min_width=640, border_mode=0) # Relleno de
    bordes (Padding negro estático) hasta un formato cuadrado de 640x640
])

count_total = 0 # Contador analítico de imágenes sintéticas exportadas con éxito

# Procesamiento iterativo de los archivos gráficos en el directorio de entrada
for filename in os.listdir(input_folder):
    if filename.lower().endswith(('.jpg', '.jpeg', '.png', 'bmp')):
        image_path = os.path.join(input_folder, filename)

        # Lectura de la matriz de la imagen en formato nativo BGR mediante OpenCV
        image = cv2.imread(image_path)

        # Control de excepciones frente a archivos corruptos o incompatibilidades
        de lectura
        if image is None:
            continue

        # Transposición de espacios de color: conversión obligatoria de BGR a RGB
        para compatibilidad con Albumentations
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        # Subbucle iterativo para la generación masiva de variantes aumentadas
        for i in range(num_augmented):
            # Invocación de la tubería probabilística sobre el tensor de la
            imagen original
            augmented = transform(image=image)['image']

            # Control de estabilidad del proceso estocástico
            if augmented is None:
                print(f" Falló la generación aumentada de {filename} (iteración {i}
                    })")
                continue

        # Reversión del espacio de color: reconversión de RGB a BGR para
        exportación estándar vía OpenCV

```

```

augmented_bgr = cv2.cvtColor(augmented, cv2.COLOR_RGB2BGR)

# Codificación sistemática del nuevo nombre de archivo indexado
new_filename = f"{os.path.splitext(filename)[0]}_aug_{i}.jpg"
out_path = os.path.join(output_folder, new_filename)

# Volcado físico y compresión del nuevo subtensor gráfico en el
# almacenamiento local
success = cv2.imwrite(out_path, augmented_bgr)
if success:
    count_total += 1
else:
    print(f" Error de escritura: No se pudo almacenar {new_filename}")

print(f" Data Augmentation finalizado. Se han incorporado con éxito {count_total}
      } nuevas muestras al ecosistema.")

```

Una vez tenemos los dos YOLOs entrenados pasamos a la extracción de las predicciones de "mal estado" de las imágenes de nuestra primera metodología con el siguiente código:

Listado 6.2: Código para extraer las naranjas en mal estado de nuestra primera metodología

```

# =====
# CÓDIGO PARA EXTRAER LAS NARANJAS MALAS PARA KERAS
# PROYECTO: Estudio y aplicación de YOLO, redes convolucionales (CNN) y modelos
#           de lenguaje (LLM) para la detección, clasificación y tratamiento de
#           naranjas
# =====

import os
import cv2

# === CONFIGURACIÓN DE ENTORNOS Y RUTAS ABSOLUTAS ===
# Directorio raíz que aloja las imágenes originales del dataset
carpeta_predicciones = 'C:/Users/IVIA/Desktop/TFG Jacobo/Datasets/TFG_Naranjas.v9-2800-buenas-2800-malas.yolov11/train/images'
# Directorio homólogo que contiene las etiquetas e inferencias en formato
# estructurado .txt
carpeta_labels = 'C:/Users/IVIA/Desktop/TFG Jacobo/Datasets/TFG_Naranjas.v9-2800-buenas-2800-malas.yolov11/train/labels'
# Indexación numérica que identifica unívocamente la clase "mal estado" en la
# configuración YOLO
clase_mala_id = 1
# Directorio destino optimizado para almacenar los subtensores correspondientes
# a los frutos afectados
carpeta_salida = 'C:/Users/IVIA/Desktop/TFG Jacobo/Pruebas/predicciones/predict_malas_512_enfermedades'

# Garantizar la existencia física del directorio de destino antes del volcado de
# imágenes

```

```
os.makedirs(carpeta_salida, exist_ok=True)

# Procesamiento iterativo de los descriptores de anotación (.txt) en el
# subconjunto de etiquetas
for archivo in os.listdir(carpeta_labels):
    if archivo.endswith(".txt"):
        # Descomposición del nombre del archivo para emparejar el fichero .txt
        # con su matriz .jpg correspondiente
        nombre_base = os.path.splitext(archivo)[0]
        ruta_imagen = os.path.join(carpeta_predicciones, nombre_base + ".jpg")
        ruta_txt = os.path.join(carpeta_labels, archivo)

        # Control de excepciones: omitir la iteración si el descriptor carece de
        # su matriz gráfica asociada
        if not os.path.exists(ruta_imagen):
            continue

        # Lectura y decodificación de la imagen digital mediante OpenCV
        imagen = cv2.imread(ruta_imagen)
        altura, ancho, _ = imagen.shape # Extracción de las dimensiones de
        # resolución nativa de la muestra

        # Apertura y lectura secuencial de las líneas del archivo de anotaciones
        # YOLO
        with open(ruta_txt, "r") as f:
            for i, linea in enumerate(f):
                partes = linea.strip().split()
                # Validación de la integridad estructural del vector de datos (
                # Clase, X, Y, W, H)
                if len(partes) < 5:
                    continue

                # Filtrado lógico: discriminar y omitir instancias pertenecientes
                # a clases no patológicas
                clase_id = int(partes[0])
                if clase_id != clase_mala_id:
                    continue

                # Desnormalización matemática de las coordenadas relativas
                # proporcionadas por YOLO
                # Convierte los centroides y dimensiones porcentuales en píxeles
                # absolutos
                x_center = float(partes[1]) * ancho
                y_center = float(partes[2]) * altura
                w = float(partes[3]) * ancho
                h = float(partes[4]) * altura

                # Cálculo de vértices geométricos espaciales (Coordenadas extremas
                # de la caja de delimitación)
                x1 = int(x_center - w / 2)
                y1 = int(y_center - h / 2)
                x2 = int(x_center + w / 2)
```

```

y2 = int(y_center + h / 2)

# Tratamiento de contorno y normalización de bordes: asegura que
# el recorte no desborde la imagen
x1, y1 = max(0, x1), max(0, y1)
x2, y2 = min(ancho, x2), min(altura, y2)

# Segmentación matricial (Slicing) de la región de interés (ROI) y
# guardado en disco
crop = imagen[y1:y2, x1:x2]
nombre_crop = f"{nombre_base}_mala_{i}.jpg"
cv2.imwrite(os.path.join(carpeta_salida, nombre_crop), crop)

print(f" Extracción completada. Subtensores de frutos patológicos almacenados
      con éxito en: {carpeta_salida}")

```

Finalmente, una vez realizado el proceso de extracción de las naranjas en mal estado procedemos al entrenamiento del Keras, donde su dataset corresponde a la lista de carpetas creadas anteriormente.

6.1.5. Tratamiento de enfermedades en las naranjas

Una vez tenemos entrenados los tres modelos que corresponden a los dos modelos YOLO y al Keras entonces ya podemos pasar al último paso de tratamiento de enfermedades.

Listado 6.3: Código para nuestro uso del LLM de Gemini para tratamiento de enfermedades de la naranja

```

# =====
# CÓDIGO DE USO DEL LLM GEMINI
# PROYECTO: Estudio y aplicación de YOLO, redes convolucionales (CNN) y modelos
#           de lenguaje (LLM) para la detección, clasificación y tratamiento de
#           naranjas
# =====

import os
import google.generativeai as genai
from dotenv import load_dotenv

# === CONFIGURACIÓN DE CREDENCIALES Y PROTOCOLOS DE SEGURIDAD ===
# Carga asíncrona de las variables de entorno locales (.env)
load_dotenv()
# Extracción de la clave de autenticación para los servicios de Google AI
API_KEY = os.getenv("GOOGLE_API_KEY")
genai.configure(api_key=API_KEY)

# INSTANCIACIÓN DEL MODELO DE FRONTERA MULTIMODAL
# Modelo de arquitectura Gemini 2.5 Flash
model = genai.GenerativeModel("gemini-2.5-flash")

# Ruta absoluta del subtensor (crop) de la naranja patológica a evaluar por el
# LLM

```

```

image_path = "C:/Users/jacob/Desktop/TFG Jacobo/Pruebas/predicciones/
predict_malas_128_enfermedades_2/otros_defectos/1
_CAM1_33997929638_0_aug_0_jpg.rf.b333c75be388a7f049631d8f2c9d6c04_mala_0.jpg
"

# LECTURA DEL ARCHIVO DE LA IMAGEN (Añadido para que exista 'image_bytes')
with open(image_path, "rb") as img_file:
    image_bytes = img_file.read()

# INGENIERÍA DE PROMPTS (PROMPT ENGINEERING)
# Escribir el prompt que sea
prompt_base = """
Eres un experto en clasificación de defectos en naranjas.
Las categorías posibles son:
1. Manchas negras
2. Hongo
3. Canker
4. Scab
5. Otros defectos
6. Trips
7. Greening
En este caso es una naranja donde presenta otros defectos.
Para cada imagen:
- Recomienda tratamiento (recuperar, procesar o descartar)
"""

# CONTROL DE EXCEPCIONES Y FLUJO DE INFERENCIA
try:
    # Ejecución de la consulta multimodal inyectando el tensor binario y las
    # directrices textuales
    response = model.generate_content(
        [
            {"mime_type": "image/jpeg", "data": image_bytes}, # Estructuración de
                los metadatos de la imagen
            prompt_base # Contexto e
                instrucciones lógicas
        ]
    )
    # Volcado en consola de la respuesta cualitativa generada por el modelo de
    # lenguaje
    print(response.text)
except Exception as e:
    # Captura de errores de red, cuotas de API o problemas de codificación de
    # archivos
    print("Error crítico durante la inferencia multimodal:", e)

```

6.2. Pruebas

En este apartado se exponen de forma sistemática los resultados obtenidos tras someter a evaluación las distintas arquitecturas.

- **Primer Bloque (Primera Metodología - YOLOv11):** Se analizan las métricas de rendimiento del primer modelo YOLO, encargado de la fase crítica de detección y discriminación binaria primaria (frutos en buen o mal estado).
- **Segundo Bloque (Primera Metodología - Keras/TensorFlow):** Se evalúa el comportamiento del Keras, el cual procesa los subtensores extraídos para realizar el diagnóstico granular de las patologías.
- **Tercer Bloque (Segunda Metodología - Segundo YOLOv11):** Se presentan los resultados del modelo YOLO enfocado en la detección multiclase directa. En este subapartado se incluye una falta de cantidad de pruebas debido a que el ajuste de hiperparámetros, el testeo de configuraciones y el data augmentation de las imágenes se derivan de la base del primer YOLO.

6.2.1. Pruebas Primera Metodología - YOLOv11

- **Referencia de validación y volumen de datos:** Para la evaluación de los modelos, se han tomado como referencia exclusiva las matrices de confusión del conjunto de validación. Por este motivo, el volumen de etiquetas analizadas es inferior al número total de imágenes del *dataset*. Por otro lado, la validación garantiza una comparación equitativa y objetiva entre las arquitecturas. Al utilizar datos que los modelos no han procesado durante la fase de entrenamiento, este conjunto actúa como un escenario operativo real. De este modo, se evalúa la capacidad de generalización de los modelos y se previene el sesgo por sobreajuste (*overfitting*), asegurando que la comparativa refleje el rendimiento potencial en un entorno de producción.
- **Criterio de evaluación prioritario:** Se ha establecido el *recall* (exhaustividad) como la métrica principal para la valoración y comparación del rendimiento de los modelos, priorizando la minimización de los falsos negativos.
- **Selección de experimentos:** Debido a la extensión del proyecto, en este documento se exponen únicamente las pruebas principales y de mayor relevancia. El histórico completo con el resto de experimentos realizados se encuentra disponible y accesible en el [repositorio de GitHub](#).

En nuestra primera prueba se hizo con un total de 742 imágenes, corresponde a la v2 del primer dataset. En este dataset se presentaban más naranjas en buen estado que mal estado. En validación, la cantidad de etiquetas en buen estado era de 521 en buen estado y 113 en mal estado. También se probaron los parámetros iniciales mostrados abajo:

Listado 6.4: Código hiperparámetros prueba 1

```
model.train(  
    data=yaml_path,  
    epochs=300,  
    imgsz=1000,
```

```

optimizer='Adam',
batch=32,
lr0=0.0005,
lrf=0.01,
momentum=0.937,
weight_decay=0.001,
device=0,
workers=8,
patience=50,
cos_lr=True,
amp=True,
val=True,
cache=True,
verbose=True,
)

```

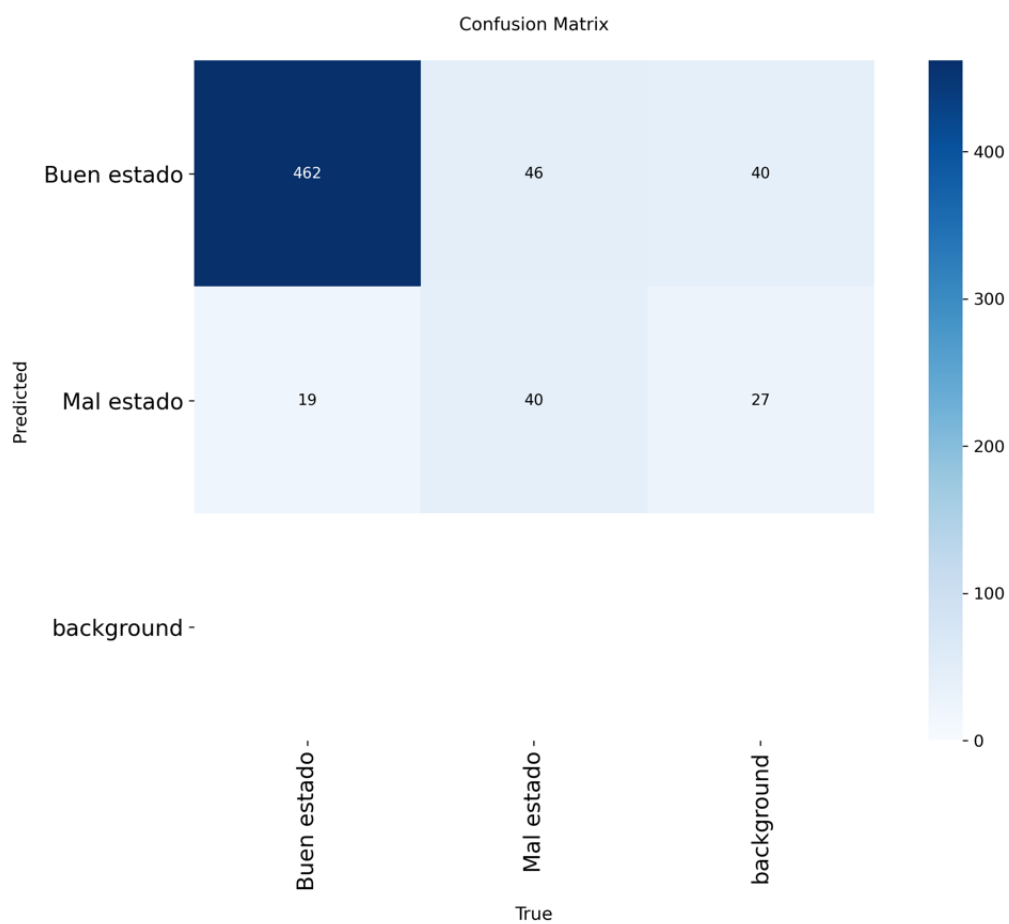


Figura 6.5: MATRIZ DE CONFUSIÓN VALIDACIÓN PRUEBA 1

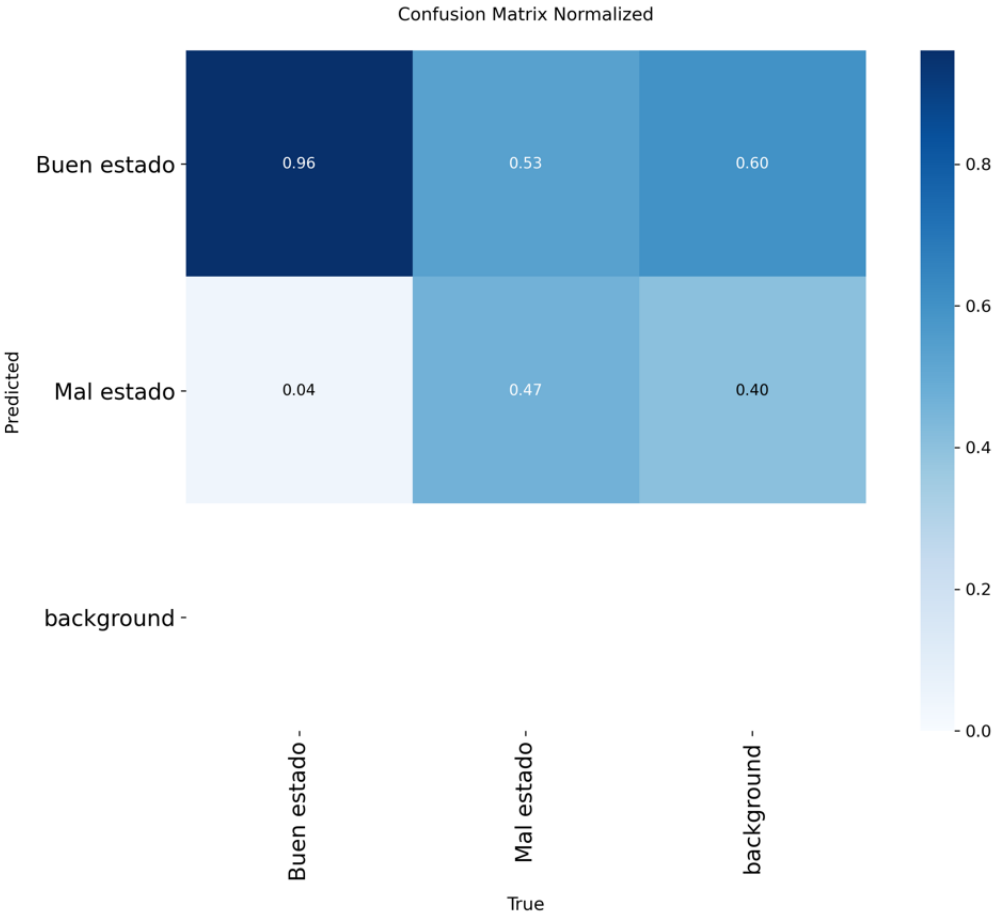


Figura 6.6: PRUEBA 1 PREDECIDAS

Dados los primeros resultados se amplificó el número de imágenes para calibrar esa diferencia de recall entre las buenas y las malas, en nuestra segunda prueba se aumentó el número de imágenes a 1160, corresponde a la v4 del primer dataset. En validación, este dataset se presentaban todavía más naranjas en buen estado que mal estado. La cantidad de etiquetas en buen estado era de 674 en buen estado y 273 en mal estado. También se cambió el `imgsz`:

Listado 6.5: Código hiperparámetros prueba 2

```
model.train(  
    data=yaml_path,  
    epochs=300,  
    imgsz=800,  
    optimizer='Adam',  
    batch=32,  
    lr0=0.0005,  
    lrf=0.01,  
    momentum=0.937,  
    weight_decay=0.001,  
    device=0,  
    workers=8,  
    patience=50,  
    cos_lr=True,  
    amp=True,  
    val=True,  
    cache=True,  
    verbose=True,  
)
```

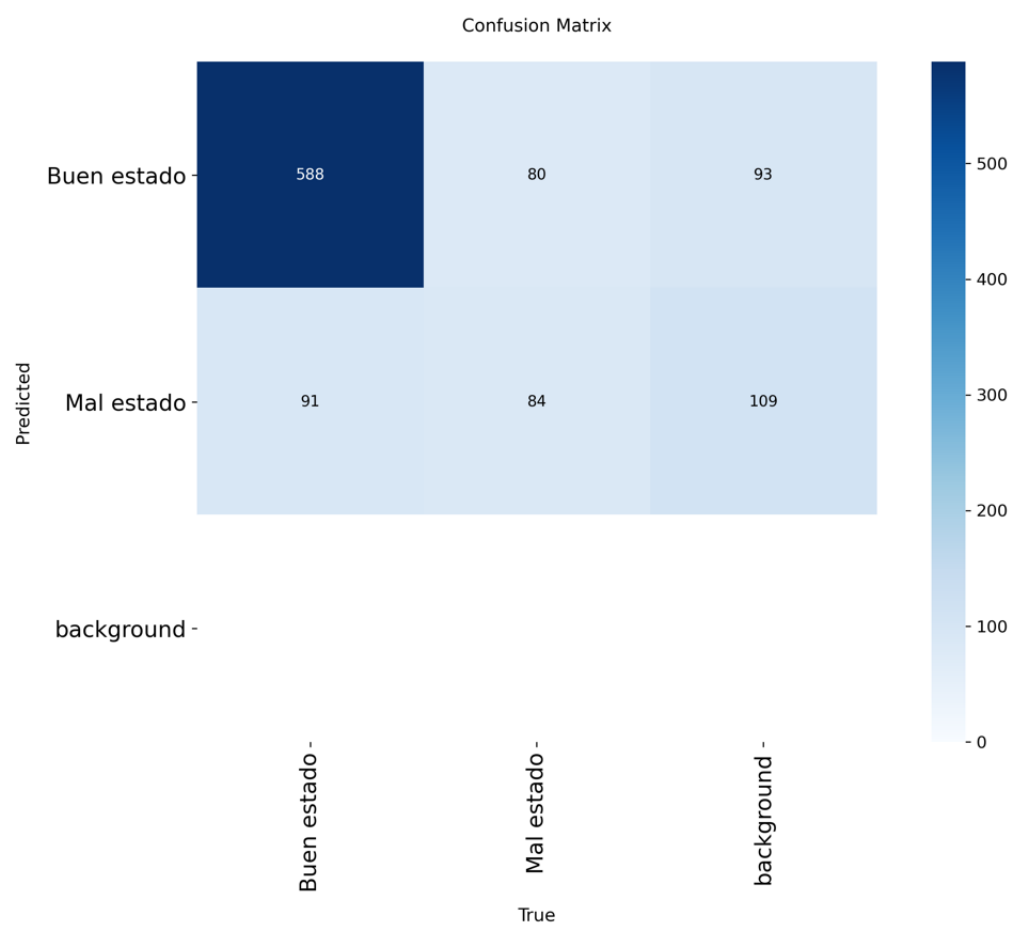


Figura 6.7: MATRIZ DE CONFUSIÓN VALIDACIÓN PRUEBA 2

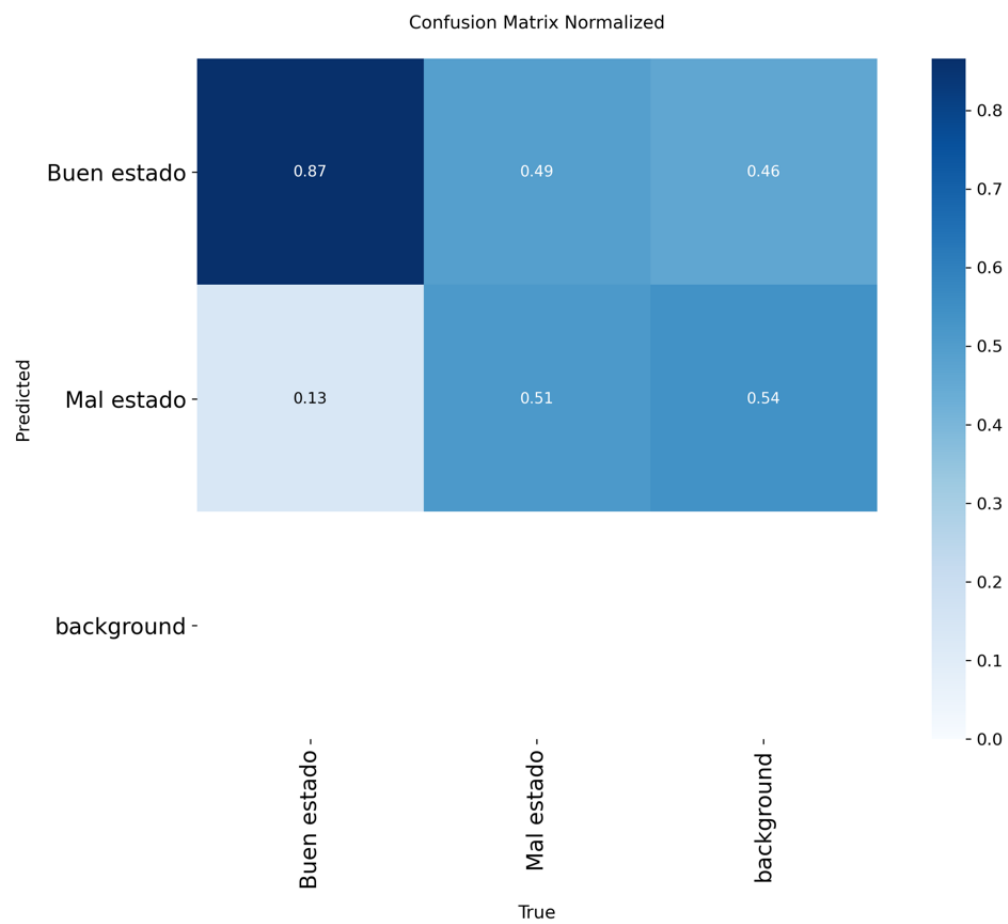


Figura 6.8: PRUEBA 2 PREDECIDAS

Dados los segundos resultados se amplificó el número de imágenes para seguir reduciendo esa diferencia de recall entre las buenas y las malas, en nuestra tercera prueba se redujo el número de imágenes a 1080, sin embargo, de esas 1080 aumentamos el número de naranjas en mal estado y reducimos el número de naranjas en buen estado, corresponde a la v6 del primer dataset. En validación, este dataset se presentaban más equilibrio entre clases. La cantidad de etiquetas en buen estado era de 538 en buen estado y 331 en mal estado.

Aquí se cambió el `imgsz` a 1000 y el `batch` a 32 para ver si estos parámetros los podía soportar nuestra gráfica:

Listado 6.6: Código hiperparámetros prueba 3

```
model.train(  
    data=yaml_path,  
    epochs=300,  
    imgsz=1000,  
    optimizer='Adam',  
    batch=32,  
    lr0=0.0005,  
    lrf=0.01,  
    momentum=0.937,  
    weight_decay=0.001,  
    device=0,  
    workers=8,  
    patience=50,  
    cos_lr=True,  
    amp=True,  
    val=True,  
    cache=True,  
    verbose=True,  
)
```

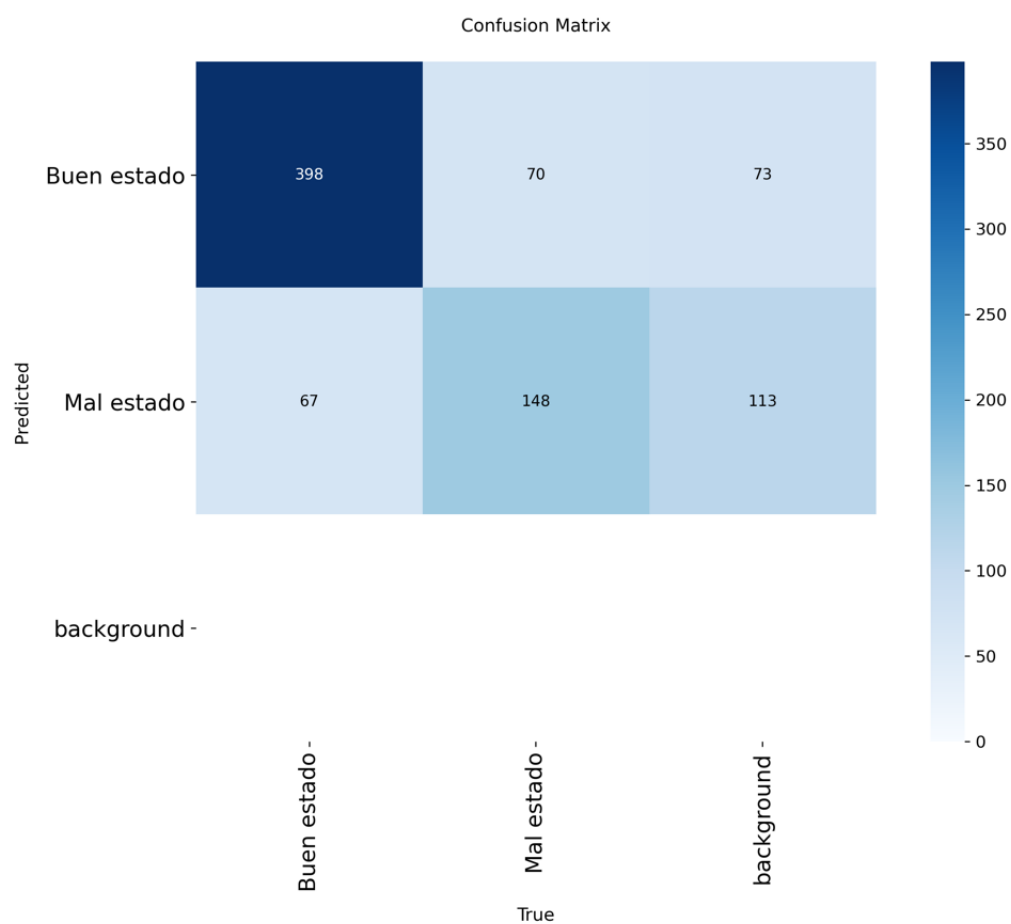


Figura 6.9: MATRIZ DE CONFUSIÓN VALIDACIÓN PRUEBA 3

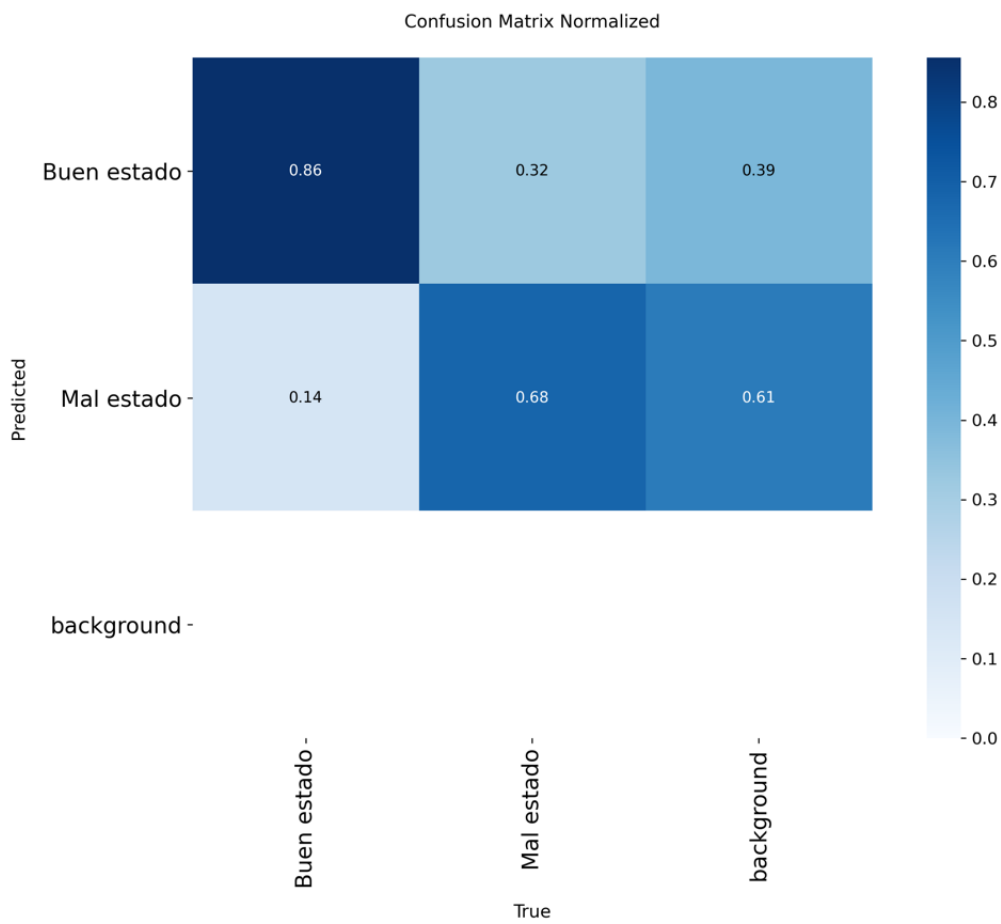


Figura 6.10: PRUEBA 3 PREDECIDAS

Dados los anteriores resultados se amplificó el número de imágenes lograr solucionar esa diferencia de recall entre las buenas y las malas, en nuestra cuarta prueba se aumentó el número de imágenes a 1861, corresponde a la última versión del primer dataset. En este dataset tenemos la misma distribución estadística entre clases en en la anterior prueba. La cantidad de etiquetas en buen estado era de 575 en buen estado y 333 en mal estado.

Listado 6.7: Código hiperparámetros prueba 4

```
model.train(  
    data=yaml_path,  
    epochs=300,  
    imgsz=800,  
    optimizer='Adam',  
    batch=24,  
    lr0=0.0005,  
    lrf=0.01,  
    weight_decay=0.0001,  
    device=0,  
    patience=70,  
    cos_lr=True,  
    amp=True,  
    val=True,  
    cache=True,  
    verbose=True,  
    augment=True,  
)
```

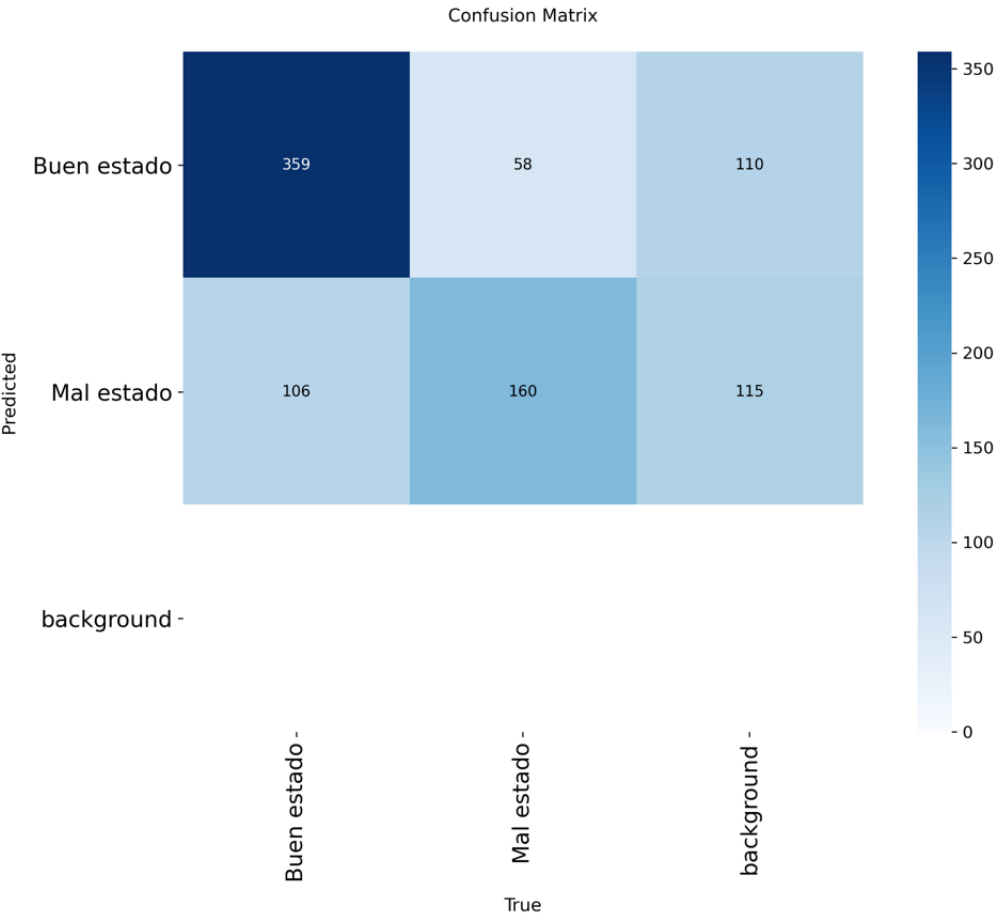


Figura 6.11: MATRIZ DE CONFUSIÓN VALIDACIÓN PRUEBA 4

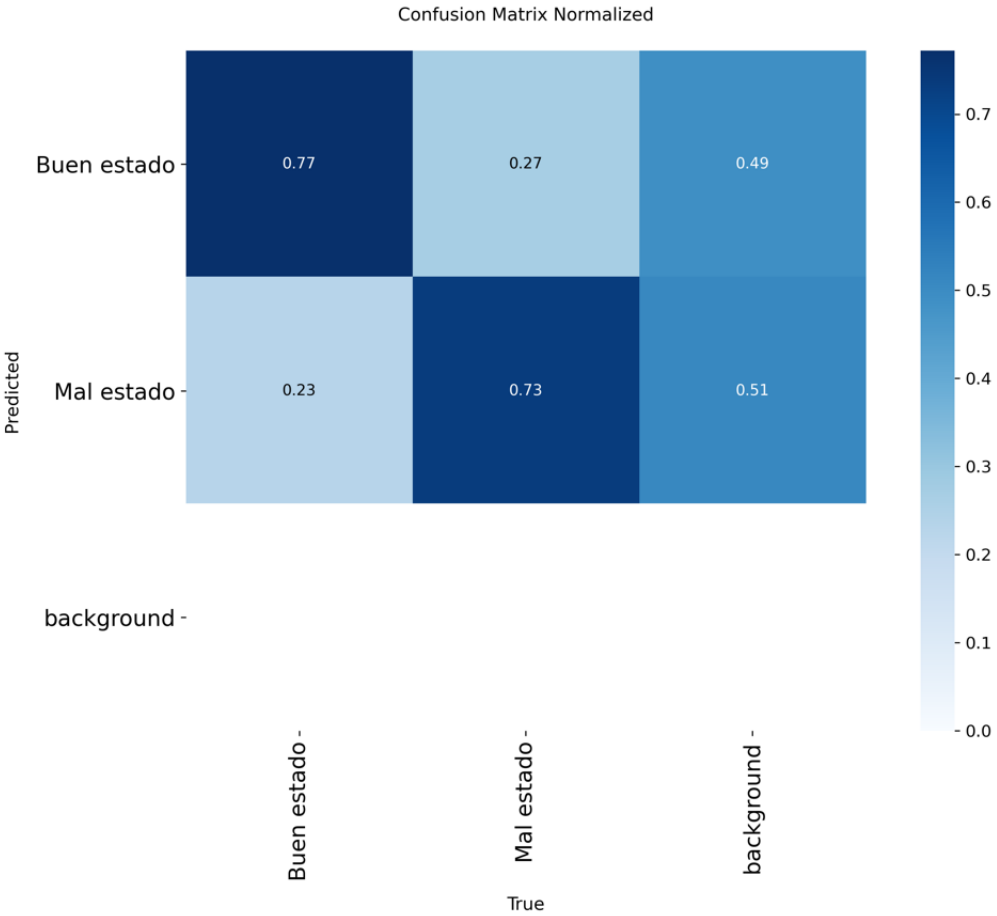


Figura 6.12: PRUEBA 4 PREDECIDAS

Dataset Final primer YOLO: 6.1 Para el resultado de la última prueba presentada mostrados en 7.1

Listado 6.8: Código hiperparámetros finales

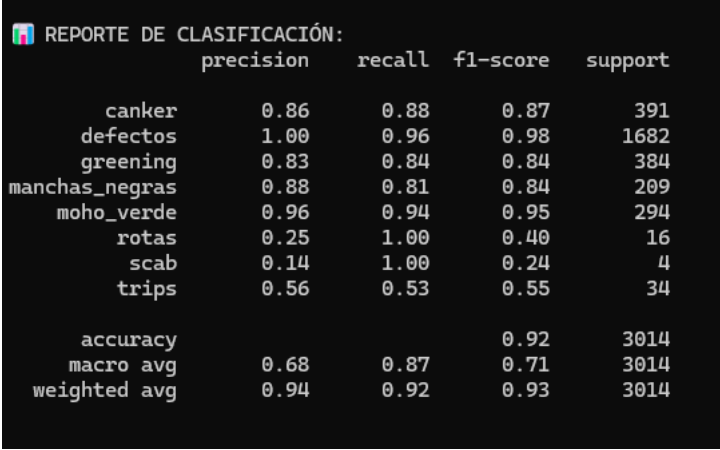
```
data=yaml_path,
epochs=200,
imgsz=600,
optimizer='Adam',
batch=-1,
lr0=0.0005,
lrf=0.01,
weight_decay=0.0001,
device=0,
patience=70,
cos_lr=True,
amp=True,
val=True,
cache=True,
verbose=True,
augment=True,
```

6.2.2. Pruebas Primera Metodología - Keras

En Keras se probó con tres resoluciones distintas de IMG SIZE dejando el mismo BATCH. En las dos primeras pruebas se hizo con 8 clases en vez de 7, es decir, que se empleó un dataset un poco distinto, la clase de otros defectos eran dos clases anteriormente: rotas y defectos. Sin embargo, sirve como prueba de que un IMG SIZE pequeño determina un peor resultado.

Listado 6.9: Hiperparámetros Primera prueba Keras

```
IMG_SIZE = 128          # Resolución dimensional (ancho y alto) para la
                        # normalización de imágenes
BATCH_SIZE = 16         # Tamaño del lote para la propagación hacia adelante
                        # y atrás
EPOCHS = 200            # Límite máximo de iteraciones globales sobre el
                        # dataset
PATIENCE = 70           # Tolerancia del Early Stopping ante el estancamiento
                        # de la función de pérdida
```



```
REPORT DE CLASIFICACIÓN:
precision    recall  f1-score   support

   canker      0.86      0.88      0.87        391
  defectos      1.00      0.96      0.98       1682
   greening      0.83      0.84      0.84        384
manchas_negras  0.88      0.81      0.84        209
   moho_verde  0.96      0.94      0.95        294
     rotas      0.25      1.00      0.40         16
     scab       0.14      1.00      0.24          4
     trips      0.56      0.53      0.55         34

 accuracy              0.92       3014
  macro avg           0.68      0.87      0.71       3014
 weighted avg           0.94      0.92      0.93       3014
```

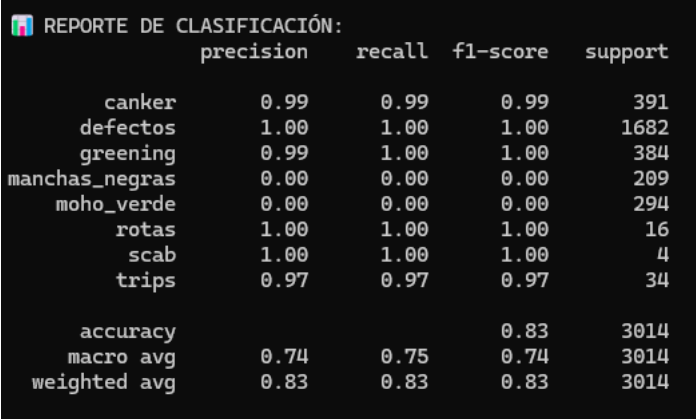
Figura 6.13: PRUEBA 1 KERAS

Listado 6.10: Hiperparámetros Segunda prueba Keras

```

IMG_SIZE = 256          # Resolución dimensional (ancho y alto) para la
                        # normalización de imágenes
BATCH_SIZE = 16         # Tamaño del lote para la propagación hacia adelante
                        # y atrás
EPOCHS = 200            # Límite máximo de iteraciones globales sobre el
                        # dataset
PATIENCE = 70           # Tolerancia del Early Stopping ante el estancamiento
                        # de la función de pérdida

```



REPORTE DE CLASIFICACIÓN:

	precision	recall	f1-score	support
canker	0.99	0.99	0.99	391
defectos	1.00	1.00	1.00	1682
greening	0.99	1.00	1.00	384
manchas_negras	0.00	0.00	0.00	209
moho_verde	0.00	0.00	0.00	294
rotas	1.00	1.00	1.00	16
scab	1.00	1.00	1.00	4
trips	0.97	0.97	0.97	34
accuracy			0.83	3014
macro avg	0.74	0.75	0.74	3014
weighted avg	0.83	0.83	0.83	3014

Figura 6.14: PRUEBA 2 KERAS

Listado 6.11: Hiperparámetros Keras finales

```

IMG_SIZE = 512          # Resolución dimensional (ancho y alto) para la
                        # normalización de imágenes
BATCH_SIZE = 16         # Tamaño del lote para la propagación hacia adelante
                        # y atrás
EPOCHS = 200            # Límite máximo de iteraciones globales sobre el
                        # dataset
PATIENCE = 70           # Tolerancia del Early Stopping ante el estancamiento
                        # de la función de pérdida

```

Dataset Final Keras: [6.2](#) Resultados de nuestra última prueba mostrados en [7.3](#)

6.2.3. Pruebas Segunda Metodología - YOLOv11

En este último apartado de pruebas hemos empleado la misma metodología que con las primeras pruebas del primer YOLO, sin embargo, al saber con que hiperparámetros entrenar exactos para este estudio resultó más corto el período de prueba. En esta primera prueba el número de imágenes era de 824, corresponde a la v3 del segundo dataset.

Listado 6.12: Código hiperparámetros prueba 1

```

model.train(
    data=yaml_path,
    epochs=200,
    imgsz=800,
    optimizer='Adam',

```

```

batch=16,
lr0=0.0005,
lrf=0.01,
weight_decay=0.0001,
device=0,
patience=70,
cos_lr=True,
amp=True,
val=True,
cache=True,
verbose=True,
augment=True,
)

```

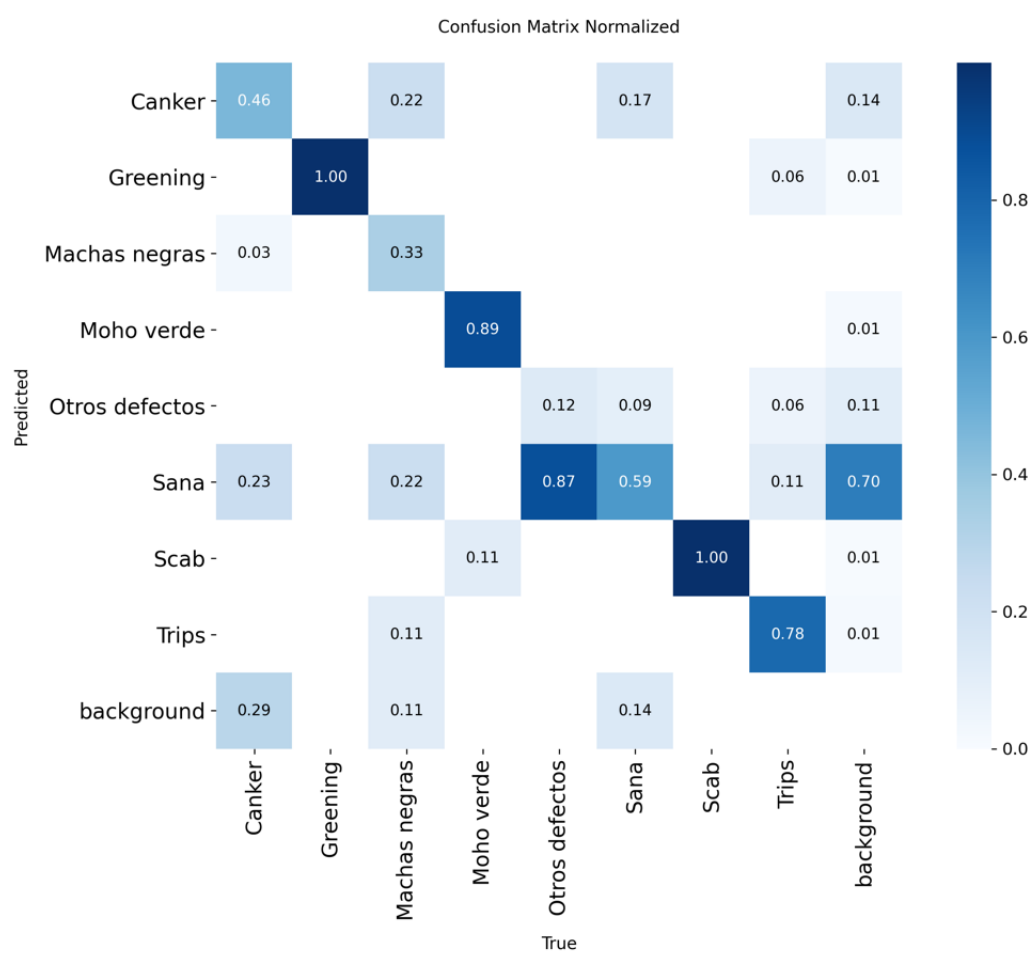


Figura 6.15: PRUEBA 1 YOLO 2

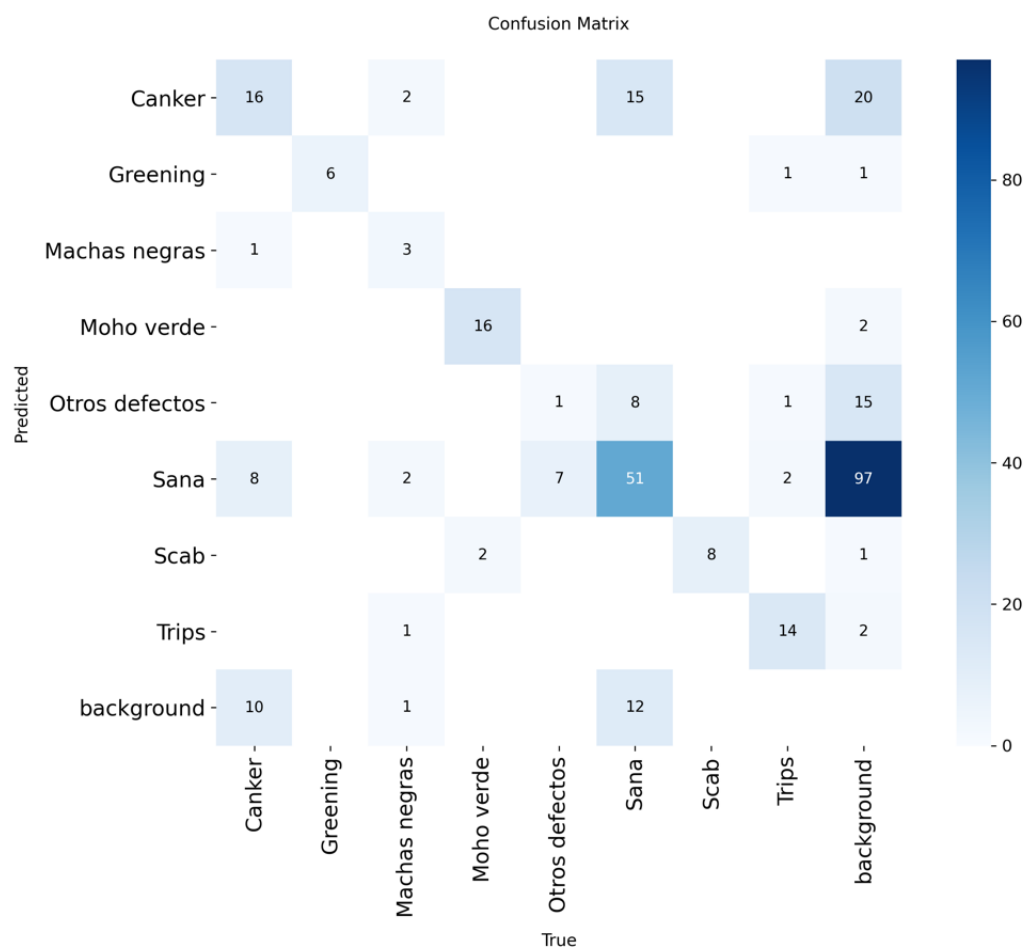


Figura 6.16: MATRIZ CONFUSIÓN 1 YOLO 2

Dados los anteriores resultados se amplificó ligeramente el número de imágenes para intentar lograr mejorar esos resultados, el número de imágenes se aumentó de 824 a 906, corresponde a la última versión del segundo dataset.

Listado 6.13: Código hiperparámetros prueba 2

```
model.train(  
    data=yaml_path,  
    epochs=200,  
    imgsz=800,  
    optimizer='Adam',  
    batch=-1,  
    lr0=0.0005,  
    lrf=0.01,  
    weight_decay=0.0001,  
    device=0,  
    patience=70,  
    cos_lr=True,  
    amp=True,  
    val=True,  
    cache=True,  
    verbose=True,  
    augment=True,  
)
```

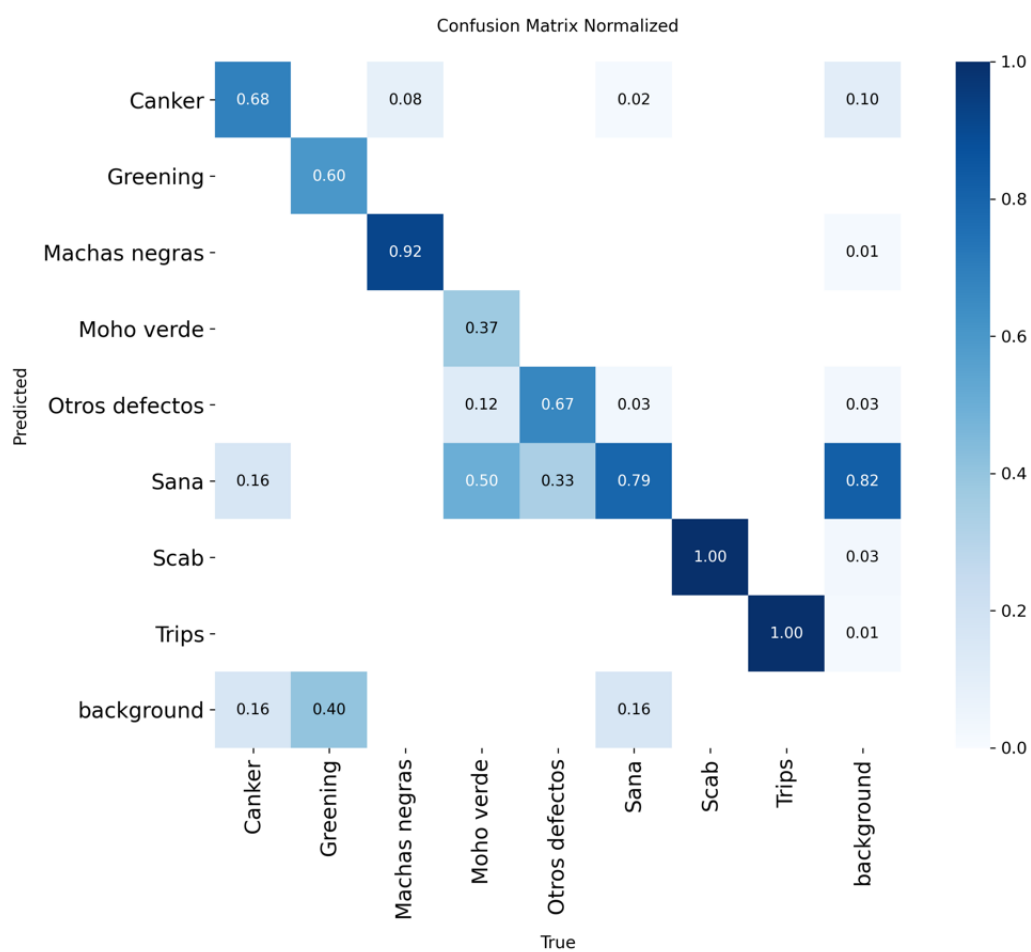


Figura 6.17: PRUEBA 2 YOLO 2

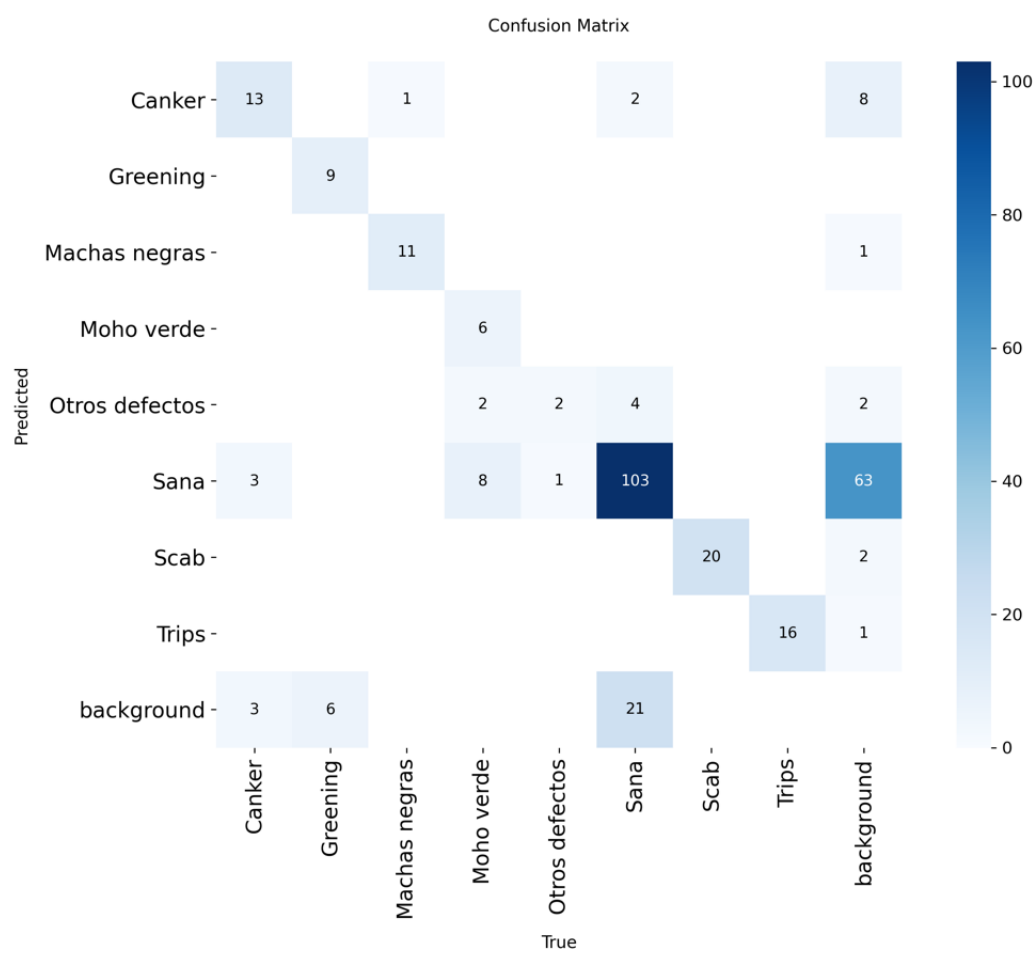


Figura 6.18: MATRIZ CONFUSIÓN 2 YOLO 2

Listado 6.14: Código hiperparámetros última prueba

```
model.train(  
    data=yaml_path,  
    epochs=200,  
    imgsz=600,  
    optimizer='Adam',  
    batch=-1,  
    lr0=0.0005,  
    lrf=0.01,  
    weight_decay=0.0001,  
    device=0,  
    patience=70,  
    cos_lr=True,  
    amp=True,  
    val=True,  
    cache=True,  
    verbose=True,  
    augment=True,  
)
```

Dataset Final segundo YOLO: [6.1](#) Resultados de nuestra última prueba mostrados en [7.4](#)

Capítulo 7

Conclusiones

7.1. Conclusiones

Con el objetivo de ofrecer una mejor visión objetiva del desempeño de este estudio, este apartado de conclusiones se ha estructurado en dos bloques diferenciados. En primer lugar, se exponen las conclusiones de carácter positivo, las cuales se presentan los resultados sobre los diferentes modelos. En segundo lugar, se detallan las conclusiones negativas, que recogen aquellas limitaciones técnicas y desviaciones respecto a las métricas esperadas. Finalmente, tenemos un resumen de las conclusiones obtenidas con lo que respecta a este apartado.

7.1.1. Conclusiones positivas

Desde una perspectiva técnica, los resultados obtenidos son sumamente satisfactorios. El modelo YOLO implementado en la primera metodología demostró un rendimiento excepcional, superando las expectativas iniciales 7.1. Por su parte, tanto el enfoque basado en Keras 7.2 y 7.3 como el segundo modelo YOLO 7.4 de la segunda metodología alcanzaron un desempeño elevado a pesar de un volumen acotado de imágenes en el conjunto de datos, las métricas de recall y demás indicadores estadísticos revelan una alta eficiencia y una excelente capacidad de generalización.

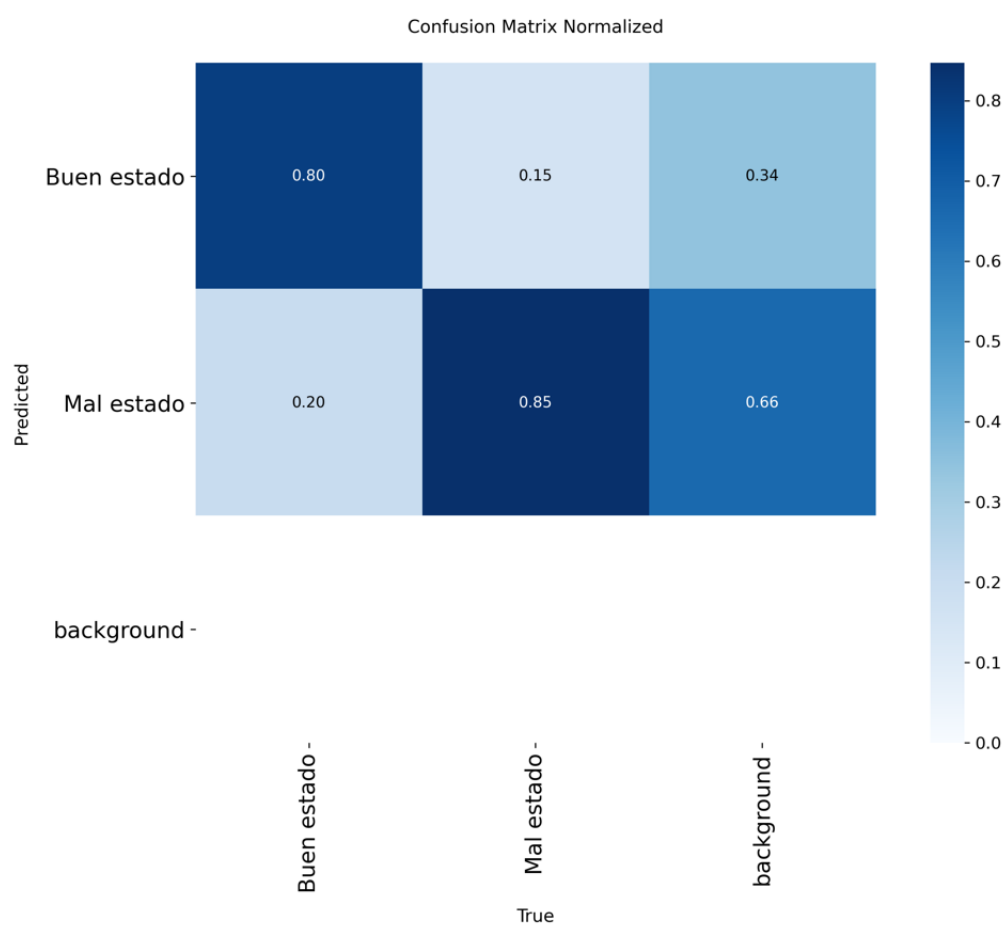


Figura 7.1: RESULTADOS FINALES PRIMER YOLO

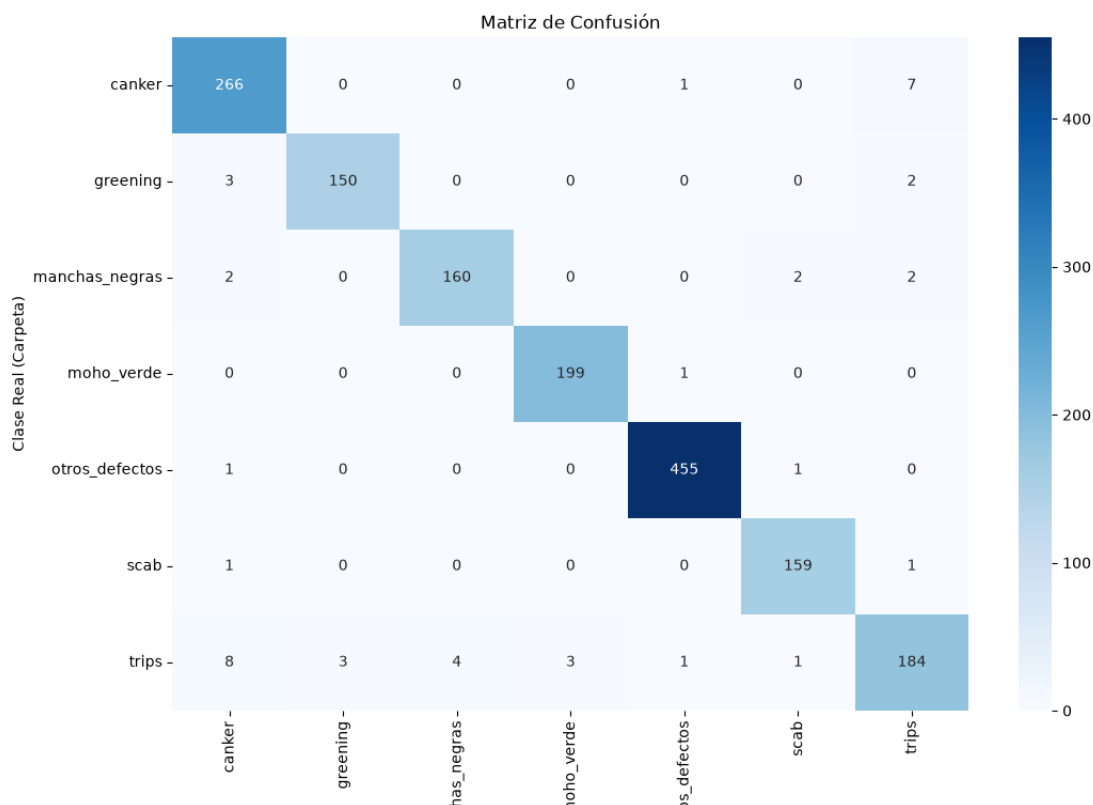


Figura 7.2: RESULTADOS FINALES KERAS

REPORTE DE CLASIFICACIÓN:				
	precision	recall	f1-score	support
canker	0.95	0.97	0.96	274
greening	0.98	0.97	0.97	155
manchas_negras	0.98	0.96	0.97	166
moho_verde	0.99	0.99	0.99	200
otros_defectos	0.99	1.00	0.99	457
scab	0.98	0.99	0.98	161
trips	0.94	0.90	0.92	204
accuracy			0.97	1617
macro avg	0.97	0.97	0.97	1617
weighted avg	0.97	0.97	0.97	1617

Figura 7.3: RESULTADOS FINALES KERAS

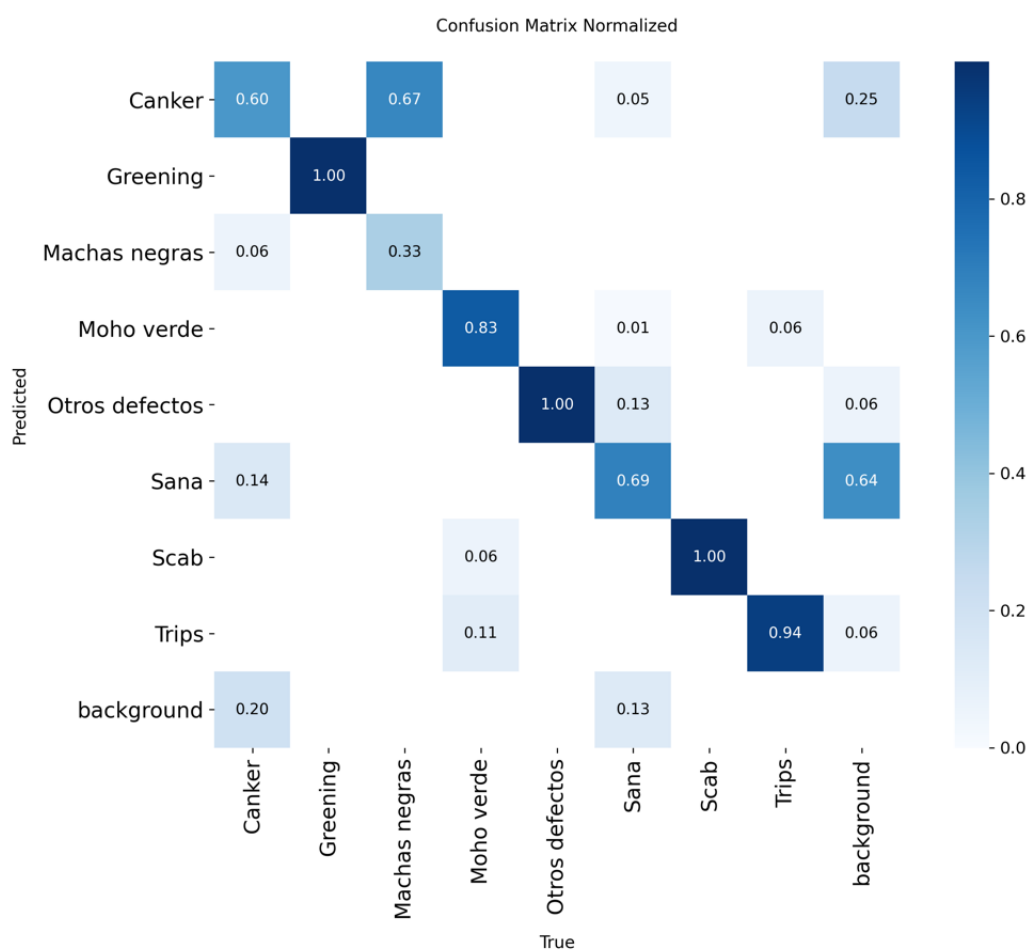


Figura 7.4: RESULTADOS FINALES SEGUNDO YOLO

7.1.2. Conclusiones negativas

El proyecto debido a la baja cantidad de imágenes de naranjas para el segundo modelo YOLO, el conjunto de datos final resultó insuficiente. Esta escasez de imágenes impactó negativamente en el entrenamiento, provocando una degradación severa en las métricas estadísticas para la clase 'Manchas Negras', la cual registró un rendimiento de 0,33 o por ejemplo también en clases como 'Sanas' con 0,69 o 'Canker' con 0,6 7.4 . Por otro lado, se constató que la obtención de métricas óptimas está sujeta a una alta demanda de tiempo y recursos computacionales. El proceso requiere una extensa fase de experimentación empírica orientada al ajuste de hiperparámetros, así como la evaluación de diversas estrategias en la distribución de los conjuntos de entrenamiento, validación y prueba (train/val/test split). Por último, el análisis del módulo basado en el modelo LLM evidenció limitaciones estructurales. Al articularse como una implementación de inferencia directa y lineal a través de la API de Gemini, el sistema hereda las mismas capacidades y restricciones operativas que una consulta convencional, es decir, que sería lo mismo que preguntarle al propio Gemini adjuntándole una imagen. Por tanto, no serviría como un LLM para tratamiento de cítricos.

```
(diagrama_gantt) C:\Users\jacob\Desktop\TFG Jacobo\Pruebas>python llm_gemini.py
C:\Users\jacob\Desktop\TFG Jacobo\Pruebas\llm_gemini.py:8: FutureWarning:
All support for the 'google.generativeai' package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the 'google.genai' package as soon as possible.
See README for more details:
https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

import google.generativeai as genai
**Clasificación:** Otros defectos

**Tratamiento:** Procesar

**Justificación:**
La naranja presenta defectos menores y superficiales que, aunque afectan su apariencia estética y podrían reducir su valor en el mercado de fruta fresca premium, no comprometen su calidad interna ni la seguridad para el consumo. Estos "otros defectos" hacen que la fruta sea ideal para ser destinada a la industria de procesamiento (producción de jugos, mermeladas, concentrados, etc.), maximizando así su aprovechamiento y evitando el descarte.
```

Figura 7.5: EJEMPLO DE LA IMPLEMENTACIÓN DEL LLM

7.1.3. Conclusiones finales

Para evaluar de forma cuantitativa el impacto y la efectividad de ambas propuestas, se procedió a un análisis comparativo basado en el (*Macro-Recall*) por clase porque el interés principal se centra en evaluar el rendimiento específico por clase. Este enfoque permite confrontar el rendimiento del ecosistema frente al diagnóstico multiclase directo bajo un mismo estándar métrico.

En la primera metodología, el rendimiento global se evalúa combinando probabilísticamente la capacidad de detección del primer modelo con la clasificación del modelo en Keras. Para ello, primero se calculan las probabilidades de sensibilidad de las enfermedades multiplicando 0.85 (los resultados obtenidos de mal estado 7.1) por el recall dada por el Keras de cada patología 7.3: (*canker* (0,97), *greening* (0,97), *manchas negras* (0,96), *moho verde* (0,99), *otros defectos* (1), *scab* (0,99) y *trips* (0,9). Los cálculos se despliegan formalmente de la siguiente manera:

$$R_{\text{Canker}} = 0,85 \times 0,97 = 0,8245 \quad (7.1)$$

$$R_{\text{Greening}} = 0,85 \times 0,97 = 0,8245 \quad (7.2)$$

$$R_{\text{Manchas negras}} = 0,85 \times 0,96 = 0,816 \quad (7.3)$$

$$R_{\text{Moho verde}} = 0,85 \times 0,99 = 0,8415 \quad (7.4)$$

$$R_{\text{Otros defectos}} = 0,85 \times 1 = 0,85 \quad (7.5)$$

$$R_{\text{Scab}} = 0,85 \times 0,99 = 0,8415 \quad (7.6)$$

$$R_{\text{Trips}} = 0,85 \times 0,9 = 0,765 \quad (7.7)$$

Teniendo estos resultados y el recall de buen estado determinada por el primer YOLO de 0,8 [7.1](#) procederíamos al cálculo final de nuestra primera metodología:

$$R_{\text{metodología 1}} = \frac{0,8245 + 0,8245 + 0,816 + 0,8415 + 0,85 + 0,8415 + 0,765 + 0,8000}{8} = 0,8204 \quad (7.8)$$

Por el contrario, en la segunda metodología, donde un único modelo YOLO asume de forma directa la tarea multiclase, el recall medio se calcula promediando los aciertos individuales de los frutos sanos y las siete patologías catalogadas: *sanas* (0,69), *canker* (0,60), *greening* (1,00), *manchas negras* (0,33), *moho verde* (0,83), *otros defectos* (1,00), *scab* (1,00) y *trips* (0,94). El operación resulta en:

$$R_{\text{metodología 2}} = \frac{0,69 + 0,60 + 1,00 + 0,33 + 0,83 + 1,00 + 1,00 + 0,94}{8} = 0,7988 \quad (7.9)$$

La ventaja matemática de la primera estrategia (0,8204 frente a 0,7988) aporta una valiosa conclusión.

A la luz de los resultados obtenidos, es posible determinar que la primera metodología de YOLO más Keras presenta un rendimiento global superior al de la segunda metodología. Este comportamiento se encuentra directamente condicionado por el volumen y el balanceo del conjunto de datos empleado durante el entrenamiento del primer modelo YOLO, lo cual dotó a la red de una mayor robustez estadística en la fase de discriminación binaria primaria. Por otro lado, si que es cierto que nuestro segundo YOLO es solamente una red neuronal que ha costado menos tiempo por cantidad de pruebas hechas y por tanto menos trabajo en el entrenamiento, entonces podemos decir que entorno a la eficiencia nuestra segunda metodología es más eficiente pero menos precisa y nuestra primera metodología es más precisa pero menos eficiente.

7.2. Trabajo futuro

Tras analizar los resultados y las conclusiones obtenidas en este estudio, se han identificado diversas áreas de oportunidad estratégicas que abren las puertas a futuras mejoras y extensiones de este trabajo.

En primer lugar, se puede plantear optimizar el rendimiento y la capacidad de generalización del segundo modelo YOLO. Para ello, es prioritario llevar a cabo una fase de expansión del dataset mediante la captura y obtención de un mayor volumen de imágenes. Asimismo, se implementará un proceso riguroso de balanceo y calibración en la distribución de las etiquetas; esto corregirá posibles sesgos en las clases minoritarias, garantizando que el modelo mantenga una alta precisión (mAP). En segundo lugar, se puede incorporar la arquitectura DETR como una tercera vía metodológica. La inclusión de este enfoque basado en transformers permitirá realizar un análisis comparativo de la misma manera frente a los métodos de una sola etapa (one-stage) representados por YOLO. De este modo, se podrá evaluar si la atención global de DETR ofrece ventajas significativas en la detección de objetos complejos o parcialmente ocluidos, enriqueciendo el marco comparativo del proyecto. Al igual también se podría comparar con la versión nueva de YOLO26 de este año para ver en que difiere con las otras metodologías. En tercer lugar, se podría perfeccionar la integración y el comportamiento del LLM de Gemini dentro del código. En lugar de utilizar llamadas lineales y estáticas a la API, el objetivo es evolucionar hacia una arquitectura basada en agentes autónomos (como sistemas Reasoning and Acting o arquitecturas multi-agente). Por último, se puede plantear también la posibilidad de migrar hacia un modelo de lenguaje de menor escala y menor cantidad de parámetros. Dado que el sistema se orientará a una tarea sumamente específica y acotada, no se requiere la capacidad generalista de un LLM como Gemini.

Bibliografía

- [1] Andreas Kamilaris and Francesc X. Prenafeta-Boldú. Deep learning in agriculture: A survey. *Computers and Electronics in Agriculture*, 147:70–90, 2018.
- [2] S. Mishra et al. Deep learning for object detection in agriculture: A review of yolo applications. *IEEE Access*, 12:45120–45145, 2024.
- [3] P. S. Thakur, P. Khanna, and H. S. Shekhawat. Vision transformers for plant disease detection: A comprehensive review. *Archives of Computational Methods in Engineering*, 31(3):1521–1545, 2024.
- [4] Joseph Nelson and Mark McQuade. Roboflow: Computer Vision Tools for Dataset Management and Model Deployment, 2020–2026. Disponible en <https://roboflow.com>.
- [5] Maxim Tkachenko, Mikhail Malyuk, Nikolai Holmanyuk, and Nikolai Liubimov. Label Studio: Data labeling software, 2020–2026. Software disponible en <https://github.com/heartexlabs/label-studio>.
- [6] Joseph Redmon, Ali Farhadi, and Glenn Jocher. YOLO (You Only Look Once): Real-Time Object Detection Framework and Architectures, 2016–2026. Ecosistema global de modelos convolucionales de una sola etapa para detección de objetos: <https://github.com/ultralytics/ultralytics>.
- [7] François Chollet et al. Keras: Deep Learning for humans, 2015–2026. Disponible en <https://keras.io> y como parte del ecosistema TensorFlow.
- [8] Salman Khan, Muzammal Naseer, Munawar Hayat, Zamir Ahmad Khan, Fahad Shahbaz Khan, and Mubarak Shah. Transformers in Vision: A Survey. *ACM Computing Surveys (CSUR)*, 54(10s):1–41, 2022.
- [9] Ayesha Hakim, Amit Kumar Srivastava, et al. Yolo-pest: An optimized yolov8x for detection of small insect pests including fruit flies. *Scientific Reports*, 15:14328, 2025. Dataset labeled using Roboflow Universe.
- [10] N. Hemalatha, Rithika Adappa, and K. M. Sreekumar. Innovative approach to cucumber pests detection: Leveraging roboflow for annotating and training. In *Artificial Intelligence: Theory and Applications (AITA 2024)*, volume 5589 of *Lecture Notes in Networks and Systems*, pages 345–358. Springer, Singapore, 2025. Uses Roboflow for dataset annotation and YOLO/COCO model training.
- [11] Google LLC. Google GenAI SDK: Official Python library for the Gemini API, 2023–2026. Disponible en <https://github.com/google/generative-ai-python>.

- [12] Tech Affiliate Team. Deepseek vs chatgpt vs gemini 2026: Which ai is better? (tested 30 days). *Tech Affiliate Blog*, 2026(1):1–15, 2025. Published December 30, 2025. Tested models: GPT-5.2, Gemini 3 Flash, DeepSeek V3.
- [13] Python Software Foundation. Python Language Reference, Version 3.11, 2022–2026. Disponible en <https://www.python.org>.
- [14] Martín Abadi, Ashish Agarwal, Paul Barham, et al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems, 2015. Software disponible en tensorflow.org.
- [15] Glenn Jocher, Jing Qiu, and Ayush Chaurasia. Ultralytics YOLO: High-Performance Real-Time Computer Vision Platform, 2023–2026. Software disponible en <https://github.com/ultralytics/ultralytics>.
- [16] Henrik Joreteg, Judy2k, and contributors. python-dotenv: Get and set dependency in the runtime environment from .env files, 2014–2026. Software disponible en <https://github.com/theofidollian/python-dotenv>.
- [17] Alexander Buslaev, Alex Parinov, Eugene Khvedchenya, Vladimir I. Iglovikov, and Alexandr A. Kalinin. Albumentations: Fast and Flexible Image Augmentations, 2020. arXiv preprint arXiv:1809.06839.
- [18] John D. Hunter. Matplotlib: A 2D Graphics Environment, 2007. Software disponible en <https://matplotlib.org>. Versión adaptada a entornos de producción en 2026.
- [19] Fabian Pedregosa, Gaël Varoquaux Equity, Alexandre Gramfort, et al. Scikit-learn: Machine Learning in Python, 2011. *Journal of Machine Learning Research*, 12, pp. 2825–2830. Versión adaptada a entornos de producción en 2026.
- [20] F. Braza et al. Fruitdetect: A deep learning app to detect fruit on camera. *GitHub / Project Report*, 2020. Implementación técnica documentada.
- [21] S. Dananjayan, Y. Tang, J. Zhuang, C. Hou, and S. Luo. Assessment of state-of-the-art deep learning based citrus disease detection techniques using annotated optical images. *Computers and Electronics in Agriculture*, 193:106658, 2022.
- [22] A. Kumar, R. Singh, and V. Patel. Smart crop advisory system: Ai-powered agricultural decision support integrating mobilenetv2 and google gemini llm. *International Journal of Agricultural and Biological Engineering*, 19(2):45–58, 2026.
- [23] Sandeep Kumar, Rajeev Ratan, and J. V. Desai. Cotton disease detection using tensorflow machine learning technique. *Advances in Multimedia*, 2022:1812025, 2022.
- [24] Rong-Zhou Qiu, Shao-Ping Chen, Mei-Xiang Chi, Rong-Bo Wang, Ting Huang, Guo-Cheng Fan, Jian Zhao, and Qi-Yong Weng. An automatic identification system for citrus greening disease (huanglongbing) using a yolo convolutional neural network and keras structures. *Frontiers in Plant Science*, 13:100260, 2022.
- [25] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, New York, NY, USA, 4th edition, 2018. Sección 1.3: Fundamentos de la representación digital de imágenes.

- [26] Osval A. Montesinos-López, Abelardo Montesinos, and Jose Crossa. Convolutional neural networks. In *Multivariate Statistical Machine Learning Methods for Genomic Prediction*, pages 533–577. Springer, 2022. License: CC BY 4.0.
- [27] Sharada P Mohanty, David P Hughes, and Marcel Salathé. Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7:1419, 2016. Sección: Image Preprocessing and Normalization (Redimensionamiento a 256x256 y normalización de inputs RGB).
- [28] Poonam Dhiman, Amandeep Kaur, V. R. Balasaraswathi, Yonis Gulzar, Ali A. Alwan, and Yasir Hamid. Image acquisition, preprocessing and classification of citrus fruit diseases: A systematic literature review. *Sustainability*, 15(12):9643, 2023.
- [29] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.
- [30] GeeksforGeeks. Geometric transformation in image processing, 2026. Disponible en <https://www.geeksforgeeks.org/electronics-engineering/geometric-transformation-in-image-processing-1>.
- [31] Xiang Wang and Kai Wang. A survey on face data augmentation for the training of deep neural networks. *Neural Computing and Applications*, 32(9):4748–4783, 2020.
- [32] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [33] Inna Logunova. Understanding backpropagation. Serokell Blog, December 2023. Accedido el 21 de junio de 2026.
- [34] Matthew Field, Nicholas Hardcastle, Michael Jameson, Noel Aherne, and Lois Holloway. Machine learning applications in radiation oncology. *Physics and Imaging in Radiation Oncology*, 19:13–24, 2021.
- [35] Rathindra Nath Mohalder, Md. Alam Hossain, and Nazmul Hossain. Classifying the supervised machine learning and comparing the performances of the algorithms. *International Journal of Advanced Research*, 12:422–438, 01 2024.
- [36] GeeksforGeeks or Towards Data Science. Supervised and unsupervised learning [differences & examples], 2024.
- [37] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(1-2):115–133, 1943.
- [38] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. 09 2017.
- [39] Hang Li. *Perceptron*, pages 39–53. 12 2023.
- [40] Akhilesh Wao and Brijesh Soni. *Performance Analysis of Sigmoid and Relu Activation Functions in Deep Neural Network*, pages 39–52. 07 2021.
- [41] Yumin Pan. Different types of neural networks and applications: Evidence from feedforward, convolutional and recurrent neural networks. *Highlights in Science, Engineering and Technology*, 85:247–255, 03 2024.

- [42] Akshay Hebbar. Mcts guided genetic algorithm for optimization of neural network weights, 2023.
- [43] David H. Hubel and Torsten N. Wiesel. Receptive fields of single neurones in the cat’s striate cortex. *The Journal of Physiology*, 148(3):574–591, 1959.
- [44] Astrid A. Zeman, James B. Ritchie, Simone Bracci, Christos Pantelis, Nikolaus Weiskopf, and Colin I. Baker. Orthogonal representations of object shape and category in deep convolutional neural networks and human visual cortex. *Scientific Reports*, 10(1), 2020.
- [45] J. S. Espinosa and M. P. Stryker. Development and plasticity of the primary visual cortex. *Neuron*, 75(2):230–249, 2012.
- [46] Anitha Pasupathy and Charles E Connor. Responses to contour features in macaque area v4. *Journal of neurophysiology*, 82(5):2490–2502, 1999.
- [47] Pulkit Agrawal, Jitendra Malik, Dustin Stansbury, and Jack Gallant. Convolutional neural networks mimic the hierarchy of visual representations in the human brain. 2017.
- [48] Stack Exchange (Data Science). The relationship between the number of filters/kernels and the number of feature maps. Data Science Stack Exchange, 2017.
- [49] Van Hiep Phung and Eun Joo Rhee. A high-accuracy model average ensemble of convolutional neural networks for classification of cloud image patches on small datasets. *Applied Sciences*, 9(21):4500, 2019. License: CC BY 4.0.
- [50] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [51] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [52] Long Ouyang, Jeffrey Jones, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.